

Fundamentos de Bases de Datos

Proyecto 3

Valentina Rojas - 202420927

Isabella Salcedo - 202420963

Maria Jose Caro - 202420310

INDICE

Modelo Relacional.....	2
Normalización.....	3
Escenarios de prueba de tipo administrador.....	7
Escenarios de prueba de tipo Cliente.....	12
ENTREGA 2.....	22
1. Análisis y revisión del modelo de datos.....	22
2. Implementación del esquema de la base de datos.....	22
3. Población de datos.....	23
4. Diseño de sentencias SQL.....	23
5. Ejecución de escenarios de prueba en APEX.....	28
6. Balance del plan de pruebas.....	38
ENTREGA 3.....	39
• Cliente - Reseñas.....	46
• Reserva - Reseñas.....	47
• Reseñas - Respuestas.....	47
Actividad 2: Diseño de sentencias MongoDB.....	49
Escenarios de prueba.....	57

Modelo Relacional

En esta justificación no vamos a explicar la razón de todos los atributos ni de todas las tablas pues priorizaremos la explicación de las relaciones dentro del modelo. Al comienzo, para representar las relaciones, cada entidad del modelo ER fue representada como una tabla independiente en el modelo relacional, donde obtuvimos tablas como Ciudades, Hoteles, Habitaciones, Tipo_Habitaciones, Usuarios, Servicios y Camas, cada una con su llave primaria (PK) y con las restricciones correspondientes.

Luego, en las relaciones de uno a muchos, incorporamos la llave foránea en la tabla que está del lado donde pueden existir varias ocurrencias asociadas a la otra. Por ejemplo, la relación entre Ciudades y Hoteles se implementó incluyendo el atributo id_ciudad como llave foránea en la tabla Hoteles, pues cada hotel pertenece a una sola ciudad y una ciudad puede tener muchos hoteles. De igual manera, en la relación entre Clientes y Reservas se agregó id_cliente como llave foránea en Reservas, porque cada reserva está asociada obligatoriamente a un cliente.

Por otro lado, para las relaciones de muchos a muchos creamos tablas intermedias, cuya llave primaria está formada por las llaves primarias de las entidades que se relacionan. Por ejemplo, en la relación entre Reservas y Servicios que se convirtió en Reserva_Servicios, se definió como llave primaria id_reserva e id_servicio, que son además llaves foráneas y también se incluyó cantidad, pues es el atributo propio de esa relación. Lo mismo ocurrió entre Tipo_habitaciones y Servicios, donde creamos la tabla

Tipo_habitaciones_Servicios y entre Tipo_habitaciones y Camas, donde surgió Tipo_habitaciones_Camas.

Finalmente, para lograr la transición a modelo relacional de Usuarios, Clientes y Administradores, decidimos hacer una tabla para cada uno. Como Clientes y Administradores heredan de Usuarios, esta última contiene los atributos comunes como nombre, apellidos y correo, mientras que las dos primeras utilizan la misma llave primaria de Usuarios, que funciona como llave foránea, con el fin de mantener la herencia representada.

Normalización

1FN:

El primer modelo relacional que realizamos no se encuentra en la primera forma normal debido a que tiene un atributo multivalor en la tabla Tipo_habitaciones, en el atributo comodidades.

Por esta razón es necesario crear otra tabla para comodidades y cómo Comodidades puede estar en muchos tipos de habitaciones y asimismo un tipo_habitacion puede tener varias comodidades, entonces hacemos una tabla intermedia de relación muchos a muchos.

Comodidades (id_comodidad, nombre_comodidad)

Tipo_habitacion-comodidad (id_tipo, id_comodidad)

Ciudades:

Ciudades (id_ciudad, nombre_ciudad)

id_ciudad → nombre_ciudad

Cumple 2FN

Cumple 3FN

Cumple BCNF

Hoteles:

Hoteles (id_hotel, nombre_hotel, direccion, telefono, descripcion, id_ciudad)

id_hotel → nombre_hotel, direccion, telefono, descripcion, id_ciudad

Cumple 2FN

Cumple 3FN

Cumple BCNF

Habitaciones:

Habitaciones (numero, id_hotel, id_tipo)

numero, id_hotel $\rightarrow$ id_tipo

Cumple 2FN

Cumple 3FN

Cumple BCNF (porque todos los atributos son determinados por toda la superllave)

Tipo_habitaciones:

Tipo_habitaciones (id_tipo, nombre_tipo, costo_alta, costo_baja, dimensiones, tipo_vista, cantidad_camras, id_hotel)

id_tipo $\rightarrow$ nombre_tipo, costo_alta, costo_baja, dimensiones, tipo_vista, cantidad_camras, id_hotel

Cumple 2FN

Cumple 3FN

Cumple BCNF

Comodidades:

Comodidades (id_comodidad, nombre_comodidad)

id_comodidad $\rightarrow$ nombre_comodidad

Cumple 2FN

Cumple 3FN

Cumple BCNF

Tipo_habitacion - Comodidad:

Tipo_habitacion - Comodidad (id_tipo, id_comodidad)

Llave: id_tipo, id_comodidad

id_tipo, id_comodidad $\rightarrow$ nada porque no hay más atributos entonces el cierre canónico da todo lo de la tabla.

Cumple 2FN

Cumple 3FN

Cumple BCNF

Usuarios:

Usuarios(id_usuario, nombre, apellidos, correo, teléfono)

Llave candidata: id_usuario

DF:

id_usuario → nombre, apellidos, correo, teléfono

Aquí, se asume que dos personas pueden poner el mismo correo, y no todos los usuarios necesariamente registran un teléfono.

2NF: Cumple, pues la llave primaria es simple, no hay dependencias parciales

3NF: Cumple, porque no hay dependencias transitivas entre atributos no llave

BCNF: Cumple porque en todas las DF el determinante es una llave candidata.

Clientes:

Clientes(id_usuario, documento_id)

Llaves candidatas: {id_usuario}, {documento_id}

DF:

id_usuario → documento_id

documento_id → id_usuario

2NF: Sí está porque al no haber atributos no primos, no pueden haber dependencias parciales.

3NF: Sí cumple, no hay dependencias entre no primos

BCNF: Sí porque en cada DF el determinante es una llave candidata.

Administradores

Administradores(id_usuario)

Al tener solo un atributo, ese es la llave candidata, y se cumple 2NF, 3NF y BCNF.

Reservas

Reservas(id_reserva, id_cliente, id_tipo, fecha_entrada, fecha_salida, adultos, niños, precio_total, estado, numero_noches, temporada)

Llave candidata: {id_reserva}

DF:

id_reserva → id_cliente, id_tipo, fecha_entrada, fecha_salida, adultos, niños, precio_total, estado, numero_noches, temporada

Al tener solo una llave candidata y solo una DF, cumple 2NF, 3NF y BCNF.

Servicios

Servicios(id_servicio, nombre, descripcion, tipo_costo, aplica_por, precio)

Llaves candidatas: {id_servicio}, {nombre}

DF:

id_servicio → nombre, descripcion, tipo_costo, aplica_por, precio

nombre → id_servicio, descripcion, tipo_costo, aplica_por, servicio

2NF: Sí cumple porque no hay ningún subconjunto de una llave que cause dependencia parcial.

3NF: Sí está en este nivel porque no hay dependencias transitivas entre no primos.

BCNF: Sí, porque los determinantes son llaves candidatas

Reserva_Servicios

Reserva_Servicios(id_reserva, id_servicio, cantidad)

Llaves candidatas: {(id_reserva, id_servicio)}

DF:

(id_reserva, id_servicio) → cantidad

2NF: Sí porque cantidad depende de toda la llave y no de una parte parcial de esta

3NF: Sí, no hay dependencias transitivas

BCNF: Sí porque todos el determinante es la llave candidata

Tipo_habitaciones_Servicios

Tipo_habitaciones_Servicios(id_tipo, id_servicio, sobrecosto, habilitado)

Llaves candidatas: {(id_tipo, id_servicio)}

DF:

(id_tipo, id_servicio) → sobrecosto, habilitado

Por las mismas razones que la anterior relación, cumple con 2NF, 3NF y BCNF.

Camas

Camas(id_cama, tipo_cama)

Llave candidata: {id_cama}

DF:

id_cama → tipo_cama

Al ser una llave simple y tratarse solo de un DF, se cumple 2NF, 3NF y BCNF.

Tipo_habitaciones_Camas

Camas(id_tipo, id_cama)

Llave candidata: {(id_tipo, id_cama)}

Como no hay atributos no primos que puedan violar las normas, se cumple con 2NF, 3NF y BCNF.

Escenarios de prueba de tipo administrador

RF1 – Registrar ciudad

RF1-E1 (Éxito) – Registrar ciudad nueva

Objetivo: Registrar una ciudad nueva válida en Ciudades.

Precondiciones: No existe una ciudad con id_ciudad='BOG'.

Datos de prueba: id_ciudad='BOG', nombre_ciudad='Bogotá'

Pasos:

- Insertar en Ciudades('BOG', 'Bogotá').
- Consultar Ciudades filtrando por id_ciudad='BOG'.

Resultado esperado: Inserción exitosa y también la ciudad aparece registrada en la consulta.

Reglas validadas: PK (id único), NN (nombre obligatorio).

RF1-E2 (Falla) – Registrar ciudad con PK repetida (unicidad)

Objetivo: Validar que el sistema no permite duplicar una ciudad por PK.

Precondiciones: Existen Ciudades('ROM','Roma').

Datos de prueba: id_ciudad='ROM', nombre_ciudad='Roma 2'

Pasos:

- Intentar insertar en Ciudades('ROM', 'Roma 2').

Resultado esperado: La inserción falla por violación de PK, luego la tabla deber quedar sin cambios.

Reglas validadas: PK (unicidad de tuplas).

RF2 - Registrar hotel

RF2-E1 (Éxito) – Registrar hotel con FK existente

Objetivo: Registrar un hotel en una ciudad previamente registrada (FK valida)

Precondiciones: Existe una ciudad Ciudades(id_ciudad = 'BOG')

Datos de prueba:

- id_hotel = 900001
- nombre_hotel = "Candelaria Suites"
- direccion = "Av. Jimenez"
- telefono = "300 111111"
- descripcion = "hotel de prueba"
- id_ciudad = "BOG"

Pasos:

- Insertar la tupla 1 en Hoteles con id id_ciudad = "BOG"

- Consultar **Hoteles** filtrando por **id_hotel = 900001**

Resultado esperado: Inserción exitosa y también el hotel queda registrado y asociado a la ciudad “BOG”.

Reglas validadas: FK (hoteles.id_ciudad en ciudades.id_ciudad), NN (campos obligatorios).

RF2-E2 (Falla) – Insertar hotel con FK inexistente (ciudad NO registrada)

Objetivo: Verificar que NO se puede registrar un hotel en una ciudad que no existe (FK inválida).

Precondiciones: No existe **Ciudades(id_ciudad='XXX')**.

Datos de prueba (tupla 1):

- **id_hotel = 900002**
- **nombre_hotel = 'Hotel Ciudad Prueba'**
- **direccion = 'Calle Falsa 123'**
- **telefono = '3111111111'**
- **descripcion = 'Hotel de prueba'**
- **id_ciudad = 'XXX'**

Pasos:

- Intentar insertar la tupla 1 en **Hoteles** con **id_ciudad='XXX'**.

Resultado esperado: La inserción falla por violación de **integridad referencial (FK)**, entonces no se crea el hotel.

Reglas validadas: FK (**Hoteles.id_ciudad** debe existir en **Ciudades**).

RF2-E3 (Éxito) – Múltiples hoteles en la misma ciudad (cardinalidad 1:N)

Objetivo: Verificar que una ciudad puede tener varios hoteles asociados.

Precondiciones: Existe **Ciudades(id_ciudad='ROM')** y ya existe un hotel con **id_ciudad='ROM'**.

Datos de prueba: Segundo hotel

- **id_hotel = 900003**
- **nombre_hotel = 'Hotel Roma Norte'**
- **direccion = 'Via Norte 456'**
- **telefono = '3222222222'**
- **descripcion = 'Segundo hotel de prueba'**
- **id_ciudad = 'ROM'**

Pasos:

- Insertar el segundo hotel en **Hoteles** con **id_ciudad='ROM'**.
- Consultar **Hoteles** por **id_ciudad='ROM'**.

Resultado esperado: Inserción exitosa. También, la consulta devuelve múltiples hoteles asociados a 'ROM'.

Reglas validadas: FK + regla de negocio/cardinalidad 1:N.

RF3 – Registrar tipo de habitación para un hotel

RF3-E1 (Éxito) – Registrar tipo de habitación completo (FK + CHECK OK)

Objetivo: Registrar un **tipo de habitación** asociado a un hotel, con costos válidos y dotación básica asociada (cama + servicios).

Precondiciones: Existe **Hoteles(id_hotel=683782)**, existe **Camas(id_cama=2, tipo_cama='King')**. Existen servicios como **Servicios(id_servicio=800010, tipo_costo='FIJO', aplica_por='RESERVA', precio>=0)** (ej.

check-in temprano) y **Servicios**(id_servicio=800011, tipo_costo='FIJO', aplica_por='PERSONA', precio>=0) (ej. desayuno)

Datos de prueba: Nuevo tipo habitación

- id_tipo=20000
- nombre_tipo='Suite Junior'
- costo_alta=5000
- costo_baja=800
- dimensiones=45
- tipo_vista='interior'
- cantidad_camas=1
- id_hotel=683782

Pasos:

- Insertar el tipo de habitación en **Tipo_habitaciones** con id_hotel=683782.
- Asociar cama al tipo: Insertar en **Tipo_habitaciones_Camas**(id_tipo=20000, id_cama=2).
- Asociar servicio por reserva al tipo: Insertar en **Tipo_habitaciones_Servicios**(id_tipo=20000, id_servicio=800010, sobrecosto=0, habilitado=1).
- Asociar servicio por persona al tipo: Insertar en **Tipo_habitaciones_Servicios**(id_tipo=20000, id_servicio=800011, sobrecosto=0, habilitado=1). Consultar **Tipo_habitaciones** y sus asociaciones (camas + servicios) para verificar.

Resultado esperado: Se crea el tipo de habitación correctamente, se crean las asociaciones de cama y servicios y finalmente queda explícito que hay servicios que aplican por **RESERVA** y otros por **PERSONA** (según **Servicios.aplica_por**).

Reglas validadas:

- **FK:** **Tipo_habitaciones.id_hotel** → **Hoteles.id_hotel**
- **FK:** **Tipo_habitaciones_Camas.id_cama** → **Camas.id_cama**
- **FK:** **Tipo_habitaciones_Servicios.id_servicio** → **Servicios.id_servicio**
- **CHECK:** rangos de costos, sobrecosto>=0, habilitado IN (0,1)

Regla de negocio: servicios por reserva vs por persona.

RF3-E2 (Falla) – Registrar tipo de habitación para hotel inexistente (FK falla)

Objetivo: Validar que no se puede registrar un tipo de habitación si el hotel no existe.

Precondiciones: No existe **Hoteles**(id_hotel=999999).

Datos de prueba:

- id_tipo=20001, nombre_tipo='Prueba', costo_alta=3000, costo_baja=700, ..., id_hotel=999999

Pasos:

- Intentar insertar el tipo en **Tipo_habitaciones** con id_hotel=999999.

Resultado esperado: La inserción falla por FK hacia **Hoteles**.

Reglas validadas: FK **Tipo_habitaciones.id_hotel**.

RF3-E3 (Falla) – Costos fuera de rango (CHECK falla)

Objetivo: Validar las restricciones de costos por temporada.

Precondiciones: Existe **Hoteles**(id_hotel=683782).

Datos de prueba:

- Insertar tipo con costo_alta=999 (violación de $1000 < costo_alta < 10000$)

- Insertar tipo con **costo_alta=15000** (*violación del límite superior*)

Pasos:

- Intentar insertar en **Tipo_habitaciones**.

Resultado esperado: Falla por CHECK de costos.

Reglas validadas: CHECK de **costo_alta**.

RF4 – Editar habitaciones

RF4-E1 (Éxito) – Editar nombre y costos del tipo de habitación (CHECK OK)

Objetivo: Editar la información de un tipo de habitación de un hotel manteniendo restricciones válidas.

Precondiciones: Existe **Tipo_habitaciones(id_tipo=10000)** asociado al hotel **id_hotel=683782**.

Datos de prueba: Cambios propuestos

- **nombre_tipo:** 'Suite Presidencial' → 'Suite Presidencial Renovada'
- **costo_alta:** 5000 → 6500
- **costo_baja:** 600 → 900

Pasos:

- Actualizar **tipo_habitaciones** para **id_tipo=10000** (solo sobre atributos permitidos por RF4).
- Consultar el registro actualizado.

Resultado esperado: La actualización se ejecuta correctamente y los nuevos valores aparecen reflejados en la consulta.

Reglas validadas: CHECK de costos + NN.

RF4-E2 (Falla) – Editar costo fuera de rango (CHECK falla)

Objetivo: Validar que no se permite actualizar con valores que violen restricciones de negocio.

Precondiciones: Existe **Tipo_habitaciones(id_tipo=10000)**.

Datos de prueba:

- **costo_alta** = 999
- **costo_alta** = 15000

Pasos:

- Intentar **actualizar el Tipo_habitaciones**

Resultado esperado: La actualización falla por violación de **CHECK** y Los valores del tipo de habitación permanecen sin cambios.

Reglas validadas: CHECK.

RF4-E3 (Éxito) – Cambiar tipo de cama y cantidad de camas (por hotel)

Objetivo: Editar tipo y cantidad de camas de un tipo de habitación.

Precondiciones: Existe **Tipo_habitaciones(id_tipo=10000, id_hotel=683782)**, existe una cama alternativa en **Camas** (ej. **id_cama=3, tipo_cama='Queen'**). (*si aún no existe, se crea primero*)

Datos de prueba:

- **cantidad_camass:** 2 → 1
- Cama: **King** → **Queen** (cambio en la asociación)

Pasos:

- Actualizar **Tipo_habitaciones.cantidad_camass** a 1 para **id_tipo=10000**.

- En **Tipo_habitaciones_Camas**, eliminar la asociación anterior (**id_tipo=10000, id_cama=2**) y agregar (**id_tipo=10000, id_cama=3**).
- Consultar **Tipo_habitaciones** y su relación con **Camas**.

Resultado esperado: La cantidad de camas queda actualizada, la cama asociada al tipo cambia correctamente.

Reglas validadas: FK hacia **Camas** (en la nueva asociación) y consistencia del modelo.

RF4-E4 (Falla) – Intentar asociar una cama inexistente (FK falla)

Objetivo: Validar que no se puede asociar un tipo de habitación a una cama que no existe.

Precondiciones: Existe **Tipo_habitaciones(id_tipo=10000)** y NO existe **Camas(id_cama=999)**.

Datos de prueba:

- insertar asociación (**id_tipo=10000, id_cama=999**)

Pasos:

- Intentar insertar en **Tipo_habitaciones_Camas(id_tipo=10000, id_cama=999)**.

Resultado esperado: Falla por FK hacia **Camas**.

Reglas validadas: FK.

RF4-E5 (Éxito) – Editar amenidades incluidas (por hotel, independiente).

Objetivo: Cambiar las amenidades incluidas en un tipo de habitación (por hotel).

Precondiciones: Existe **Tipo_habitaciones(id_tipo=10000, id_hotel=683782)** y también existen amenidades **Comodidades(id_comodidad=218, nombre_comodidad='wifi')** y otra adicional (ej. **219='TV'**).

Pasos:

- Eliminar la amenidad actual del tipo (ej. quitar wifi).
- Agregar una nueva amenidad (ej. TV).
- Consultar amenidades asociadas al **id_tipo=10000**.

Resultado esperado: La lista de amenidades queda actualizada correctamente.

Reglas validadas: FK en la tabla intermedia de tipo-comodidad (si aplica).

RF4-E6 (Éxito clave) – Independencia por hotel

Objetivo: Comprobar que editar un tipo de habitación en un hotel no afecta otros hoteles.

Precondiciones:

- Existe el hotel A: **Hoteles(id_hotel=683782)** y tiene **Tipo_habitaciones(id_tipo=10000, id_hotel=683782)**.
- Existe otro hotel B en otra ciudad (o misma ciudad): **Hoteles(id_hotel=700000)** con su propio tipo **Tipo_habitaciones(id_tipo=11000, id_hotel=700000)** (puede tener el mismo **nombre_tipo** si así lo diseñaron).

Pasos:

- Editar el tipo del hotel A (**id_tipo=10000**): cambiar **costo_alta** o **nombre_tipo**.
- Consultar el tipo del hotel B (**id_tipo=11000**) y verificar que no cambió.

Resultado esperado: Solo cambia el tipo del hotel A, también el hotel B mantiene su información intacta.

Reglas validadas: separación por FK **Tipo_habitaciones.id_hotel** (independencia por hotel).

Escenarios de prueba de tipo Cliente

RF5 – Creación de cuenta de cliente

RF5-E1 (Éxito) – Cliente crea su cuenta (registro completo Usuario + Cliente)

Objetivo: Registrar la información de un cliente: identificación, nombre, correo y teléfono.

Precondiciones: No existe `Usuarios(id_usuario=500001)`.

Datos de prueba:

Usuario:

- `id_usuario=500001`
- `nombre='Ana'`
- `apellidos='Pérez'`
- `correo='ana.perez@mail.com'`
- `telefono='3100000000'`

Cliente:

- `id_usuario=500001`
- `documento_id='CC123456789'`

Pasos:

- Insertar en `Usuarios` los datos del usuario.
- Insertar en `Clientes` la tupla con `id_usuario=500001` y `documento_id`.
- Consultar `Usuarios` y `Clientes` para verificar que quedó registrado.

Resultado esperado: Inserción exitosa en ambas tablas y el usuario aparece en `Usuarios` y también como cliente en `Clientes`.

Reglas validadas: PK `Usuarios.id_usuario`, FK `Clientes.id_usuario` → `Usuarios.id_usuario` y NN en nombre/correo/teléfono/documento_id.

RF5-E2 (Falla) – Intentar registrar cliente sin usuario previo (FK falla)

Objetivo: Validar que no se puede crear un cliente si no existe primero el usuario.

Precondiciones: No existe `Usuarios(id_usuario=700000)`.

Datos de prueba:

- Insertar en `Clientes(id_usuario=700000, documento_id='CC000')`

Pasos:

- Intentar insertar la tupla en `Clientes` sin haber insertado el usuario en `Usuarios`.

Resultado esperado: La inserción falla por integridad referencial (FK).

Reglas validadas: FK `Clientes.id_usuario` → `Usuarios.id_usuario`.

RF6 – Gestión de cuenta de cliente

RF6-E1 (Éxito) – El cliente actualiza sus datos personales

Objetivo: Permitir que el cliente modifique sus datos personales cuando lo necesite.

Precondiciones:

- Existe un usuario registrado con `id_usuario=123456`.
- Ese usuario está registrado como cliente en `Cientes (id_usuario=123456)`.

Datos de prueba (cambios):

- `telefono:` de `3123245649` a `3001234567`
- `correo:` de `jbeltran@gmail.com` a `juan.beltran@correo.com`

Pasos:

- El cliente ingresa a “gestión de cuenta”.
- Actualiza su teléfono y correo con los valores de prueba.
- Guarda los cambios.
- Se consulta la información del cliente para verificar la actualización.

Resultado esperado: Los datos quedan actualizados, también al consultar de nuevo la cuenta, se muestran los valores nuevos.

Reglas validadas: campos obligatorios (NN) y consistencia de datos.

RF6-E2 (Falla) – Un cliente intenta modificar datos de otro cliente (roles/regla de negocio)

Objetivo: Asegurar que un cliente solo pueda modificar su propia información.

Precondiciones:

- Existe el Cliente A con `id_usuario=123456`.
- Existe el Cliente B con `id_usuario=500002`.

Datos de prueba: cambiar el teléfono del Cliente B a `3333333333`.

Pasos:

- El Cliente A ingresa al sistema.
- Intenta editar la cuenta del Cliente B.
- Intenta guardar los cambios.

Resultado esperado: El sistema rechaza la operación por falta de permisos y los datos del Cliente B permanecen igual.

Reglas validadas: control de acceso por rol (regla de negocio).

RF6-E3 (Falla opcional) – Intentar dejar un campo obligatorio vacío (NN)

Objetivo: Validar que no se permite dejar datos obligatorios vacíos.

Precondiciones: Existe un cliente con `id_usuario=123456`.

Datos de prueba: dejar `correo` vacío.

Pasos:

- El cliente entra a editar su perfil.
- Borra el valor del correo y guarda.

Resultado esperado: El sistema rechaza la actualización y el correo anterior se mantiene.

Reglas validadas: restricciones de obligatoriedad (NN).

RF7 – Realizar reserva de habitación

RF7-E1 (Éxito) – Registrar reserva completa con disponibilidad + servicios

Objetivo: Registrar una reserva completa (cliente, tipo, fechas, adultos/niños, servicios y precio total) solo si hay disponibilidad.

Precondiciones:

- Existe el cliente en el sistema: **Cientes**: (id_usuario=123456, documento_id='CC123')
- Existe el tipo de habitación: **Tipo_habitaciones**: (id_tipo=10000, nombre_tipo='Suite Presidencial', id_hotel=683782, costo_alta=5000, costo_baja=600, ...)
- Existen servicios que el cliente puede seleccionar: como servicio por reserva: **Servicios**: (id_servicio=728937, nombre='Cancelación Flexible', aplica_por='RESERVA', precio=50, ...)o Servicio por persona: **Servicios**: (id_servicio=800011, nombre='Desayuno', aplica_por='PERSONA', precio=20, ...)
- Hay disponibilidad del id_tipo=10000 para el rango de fechas elegidos.

Datos de prueba:

- Reservas (id_reserva=900100, id_cliente=123456, id_tipo=10000, fecha_entrada='10/10/26', fecha_salida='20/10/26', adultos=2, niños=1, precio_total=1200, estado='ACTIVA', numero_noches=10, temporada='ALTA')
- Reserva_Servicios (servicios seleccionados para esa reserva): (id_reserva=900100, id_servicio=728937, cantidad=1) *(aplica por reserva → 1 vez)*, (id_reserva=900100, id_servicio=800011, cantidad=3) *(aplica por persona → 2 adultos + 1 niño = 3)*

Pasos:

- El cliente selecciona el tipo de habitación 10000.
- Ingresa fechas de entrada/salida y número de adultos/niños.
- El sistema verifica disponibilidad para ese tipo y esas fechas.
- El cliente selecciona servicios adicionales (uno por reserva y uno por persona).
- El sistema registra la reserva y asocia los servicios seleccionados.

Resultado esperado: La reserva queda registrada con estado **ACTIVA**, los servicios quedan asociados a la reserva con sus cantidades correctas, se cumplen las restricciones del modelo (FK y CHECK).

RF7-E2 (Falla) – Reserva rechazada por cliente inexistente (FK)

Objetivo: Validar que no se puede reservar si el cliente no existe.

Precondición: No existe **Cientes**(id_usuario=700000).

Datos de prueba:

- Reservas: (id_reserva=900101, id_cliente=700000, id_tipo=10000, fecha_entrada='10/10/26', fecha_salida='20/10/26', adultos=2, niños=0, precio_total=1000, estado='ACTIVA', numero_noches=10, temporada='ALTA')

Resultado esperado: La reserva no se registra por violación de FK hacia **Cientes**.

RF7-E3 (Falla) – Reserva rechazada por tipo de habitación inexistente (FK)

Precondición: No existe **Tipo_habitaciones**(id_tipo=99999).

Datos de prueba:

- Reservas: (id_reserva=900102, id_cliente=123456, id_tipo=99999, fecha_entrada='10/10/26', fecha_salida='20/10/26', adultos=2, niños=0, precio_total=1000, estado='ACTIVA', numero_noches=10, temporada='ALTA')

Resultado esperado: La reserva no se registra por violación de FK hacia **Tipo_habitaciones**.

RF7-E3 (Falla) – No hay disponibilidad (regla de negocio)

Precondición: Para id_tipo=10000 y fechas 10/10/26–20/10/26, no hay habitaciones disponibles.

Flujo:

- El cliente intenta reservar ese tipo y esas fechas.
- El sistema verifica disponibilidad y detecta que está en cero.

Resultado esperado: El sistema rechaza la reserva por regla de negocio “solo si hay disponibilidad” y no se crean registros de reserva ni asociaciones de servicios.

RF8 – Gestionar la reserva por parte del cliente

RF8-E1 (Éxito) – Consultar detalles de su reserva

Objetivo: El cliente consulta los detalles de su reserva.

Precondiciones:

- Existe una reserva activa del cliente: `Reservas:(id_reserva=762536, id_cliente=123456, id_tipo=10000, fecha_entrada='10/10/26', fecha_salida='20/10/26', adultos=2, niños=1, precio_total=1200, estado='ACTIVA', numero_noches=10, temporada='ALTA')`
- Existen sus servicios asociados: `Reserva_Servicios: (762536, 728937, 1)` (y otros si aplica)

Pasos:

- El cliente inicia sesión.
- Ingresa a “Mis reservas”.
- Selecciona la reserva `id_reserva=762536`.
- El sistema muestra la información de la reserva y sus servicios.

Resultado esperado: Se muestran correctamente fechas, tipo de habitación, cantidad de personas, estado, precio total y servicios asociados.

RF8-E2 (Falla) – Intentar consultar una reserva que no le pertenece (roles/regla de negocio)

Objetivo: Evitar que un cliente vea reservas de otros.

Precondiciones:

- Existe otra reserva: `Reservas: (id_reserva=800000, id_cliente=500002, ..., estado='ACTIVA')`
- El cliente autenticado es `id_cliente=123456`.

Pasos:

- El cliente 123456 intenta acceder a la reserva `id_reserva=800000`.

Resultado esperado:

- El sistema rechaza el acceso (no autorizado).
- No se muestran datos de la reserva.

RF8-E3 (Éxito) – Modificar fechas con disponibilidad

Objetivo: Cambiar las fechas de la reserva cuando hay disponibilidad.

Precondiciones:

- Existe reserva activa del cliente: `Reservas: (id_reserva=762536, id_cliente=123456, id_tipo=10000, fecha_entrada='10/10/26', fecha_salida='20/10/26', estado='ACTIVA', ...)`
- Hay disponibilidad del `id_tipo=10000` para las nuevas fechas.

Datos de prueba:

- `fecha_entrada='12/10/26'`
- `fecha_salida='22/10/26'`

Pasos:

- El cliente selecciona “Modificar fechas”.
- Ingresa las nuevas fechas.
- El sistema valida disponibilidad.
- El sistema guarda la modificación.

Resultado esperado: La reserva queda actualizada con las nuevas fechas, la reserva sigue en estado **ACTIVA**.

RF8-E4 (Falla) – Modificar fechas sin disponibilidad

Objetivo: Evitar modificaciones cuando no hay disponibilidad.

Precondiciones: Existe reserva **id_reserva=762536** (ACTIVA) y para las fechas nuevas propuestas no hay disponibilidad del **id_tipo=10000**.

Pasos:

- El cliente intenta modificar a un rango sin cupo.
- El sistema valida disponibilidad y detecta cero.

Resultado esperado: El sistema rechaza la modificación, la reserva mantiene sus fechas originales.

Escenarios de prueba para los requerimientos funcionales de consulta

RFC1 – Consultar habitaciones disponibles en un intervalo de fechas

RFC1-E1 (Éxito) – Consulta válida con resultados

Objetivo: Consultar habitaciones disponibles en un hotel específico para un rango de fechas y con ciertos criterios.

Precondiciones:

- Existe el hotel: **Hoteles** (**id_hotel=683782**, **nombre_hotel='Palazzo Navona'**, **id_ciudad='ROM'**, ...)
- Existen tipos de habitación para ese hotel: **Tipo_habitaciones** (**id_tipo=10000**, **nombre_tipo='Suite Presidencial'**, **id_hotel=683782**, **cantidad_camas=2**, **costo_alta=5000**, **costo_baja=600**, ...) (**id_tipo=20000**, **nombre_tipo='Suite Junior'**, **id_hotel=683782**, **cantidad_camas=1**, **costo_alta=3000**, **costo_baja=700**, ...)
- Existe una reserva activa que ocupa parcialmente un tipo: **Reservas** (**id_reserva=900100**, **id_cliente=123456**, **id_tipo=10000**, **fecha_entrada='10/10/26'**, **fecha_salida='20/10/26'**, **estado='ACTIVA'**, ...)
- Pero aún hay disponibilidad restante del tipo **10000** o del tipo **20000**.

Datos de prueba:

- Hotel: **id_hotel=683782**
- Fecha entrada: **12/10/26**
- Fecha salida: **15/10/26**
- Criterios: mínimo 1 cama, capacidad ≥ 2 personas, rango de precio dentro del presupuesto

Pasos:

- El usuario (sin iniciar sesión) accede al módulo de búsqueda.
- Selecciona el hotel **Palazzo Navona**.
- Ingresa fechas **12/10/26** a **15/10/26**.
- Define criterios (ej. mínimo 2 personas).

- El sistema: Filtra por hotel, filtra por fechas, excluye tipos sin disponibilidad, aplica criterios adicionales.
- Muestra los tipos de habitación disponibles.

Resultado esperado: Se muestran los tipos disponibles que cumplen que pertenecen al hotel consultado, tienen disponibilidad en el intervalo, cumplen los criterios, y no se requiere autenticación.

RFC1-E2 (Éxito) – Usuario sin cuenta puede consultar

Objetivo: Verificar que no se requiere login.

Pasos:

- Usuario accede directamente al buscador.
- Realiza consulta con fechas válidas.
- El sistema muestra resultados.

Resultado esperado: La consulta funciona sin autenticación y no se genera error por falta de sesión.

RFC1-E3 (Falla) – No hay disponibilidad en el rango

Precondiciones:

- Para el hotel 683782 y el rango 10/10/26–20/10/26:
- Todas las habitaciones del tipo 10000 están ocupadas.
- No existen otros tipos con disponibilidad.

Datos de prueba:

- Id_hotel = 683782
- Fechas: 12/10/26 – 15/10/26

Resultado esperado: El sistema muestra: “No hay habitaciones disponibles para las fechas seleccionadas.” No devuelve tipos ocupados.

RFC1-E4 (Falla lógica) – Fechas inválidas

Datos de prueba

- Fecha entrada: 20/10/26
- Fecha salida: 15/10/26 (salida anterior a entrada)

Resultado esperado: El sistema rechaza la consulta y muestra mensaje de error indicando que el intervalo de fechas es inválido.

RFC2 – Consultar reservas a través del tiempo

RFC2-E1 (Éxito) – Consulta general por intervalo de tiempo

Objetivo: Visualizar la evolución de reservas en un intervalo determinado.

Precondiciones:

- **Hoteles** (id_hotel=683782, nombre_hotel='Palazzo Navona', id_ciudad='ROM')
- **Reservas** (id_reserva=900100, id_cliente=123456, id_tipo=10000, fecha_entrada='10/01/26', estado='FINALIZADA', ...)

Datos de prueba:

- Intervalo: Desde: 01/01/26, hasta: 31/03/26

Pasos:

- El administrador ingresa al módulo de reportes.

- Selecciona intervalo de fechas 01/01/26 – 31/03/26.
- Ejecuta la consulta.

Resultado esperado: El sistema muestra: Número de reservas por mes (enero, febrero, marzo) y tendencia o conteo de flujo. También, las reservas CANCELADAS pueden excluirse del conteo (según regla definida por ustedes).

RFC2-E2 (Éxito) – Filtrar por hotel específico

Objetivo: Ver evolución de reservas solo para un hotel.

Datos de prueba:

- Intervalo: 01/01/26 – 31/03/26
- Filtro: id_hotel=683782

Pasos:

- Seleccionar intervalo.
- Aplicar filtro por hotel.
- Ejecutar consulta.

Resultado esperado: Solo se contabilizan reservas asociadas a Tipo_habitaciones cuyo id_hotel=683782 y no aparecen reservas del hotel 700000.

RFC2-E3 (Éxito) – Filtrar por ciudad

Objetivo: Ver flujo de reservas por ciudad.

Precondición: Existe Ciudades('ROM') y Ciudades('BOG').

Datos de prueba:

- Intervalo: 01/01/26 – 31/03/26
- Filtro: id_ciudad='ROM'

Resultado esperado: Solo se muestran reservas de hoteles ubicados en Roma y no aparecen reservas de hoteles en Bogotá.

RFC2-E4 (Sin resultados) – Intervalo sin reservas

Datos de prueba:

- Intervalo: 01/01/25 – 31/01/25
- No existen reservas en ese rango.

Resultado esperado: El sistema muestra “no existen reservas en el intervalo seleccionado.” No genera error.

RFC2-E5 (Falla lógica) – Intervalo inválido

Datos de prueba:

- Desde: 31/03/26
- Hasta: 01/01/26

Resultado esperado: El sistema rechaza la consulta y muestra mensaje de error por rango de fechas inválido.

RFC3 – Consultar cantidad de huéspedes por fechas y ciudad

RFC3-E1 (Éxito) – Total de huéspedes por ciudad en un rango de fechas

Objetivo: Obtener el número total de huéspedes (adultos + niños) para una ciudad en fechas dadas, sumando todos los hoteles de esa ciudad.

Precondiciones:

- Ciudades: (id_ciudad='ROM', nombre_ciudad='Roma')
- Hoteles: (id_hotel=683782, nombre_hotel='Palazzo Navona', id_ciudad='ROM', ...), (id_hotel=683783, nombre_hotel='Roma Central', id_ciudad='ROM', ...)
- Tipo_habitaciones: (id_tipo=10000, id_hotel=683782, ...), (id_tipo=11000, id_hotel=683783, ...)
- Reservas (en el rango consultado y no canceladas)

Datos de prueba:

- Ciudad: id_ciudad='ROM'
- Rango de fechas: 10/10/26 – 12/10/26

Pasos:

- El administrador entra al reporte “Huéspedes por ciudad”.
- Selecciona la ciudad ROM.
- Define el rango de fechas 10/10/26 – 12/10/26.
- Ejecuta la consulta.
- El sistema suma huéspedes por reservas que se cruzan con ese rango y pertenecen a hoteles en esa ciudad.

Resultado esperado: El sistema muestra el total de huéspedes en Roma en ese rango.

RFC3-E2 (Éxito) – Filtrar por hotel específico dentro de la ciudad

Objetivo: Ver huéspedes en un hotel específico (dentro de una ciudad).

Datos de prueba:

- Ciudad: id_ciudad='ROM'
- Hotel: id_hotel=683782
- Rango de fechas: 10/10/26 – 12/10/26

Pasos:

- Administrador selecciona ciudad ROM.
- Activa el filtro de hotel y elige 683782.
- Ejecuta la consulta.

Resultado esperado: El total incluye solo reservas asociadas al hotel 683782.

RFC3-E3 (Sin resultados) – Ciudad sin reservas en el rango

Precondiciones: Existe la ciudad, pero no hay reservas que coincidan con las fechas.

Datos de prueba:

- Ciudad: ROM
- Rango: 01/01/25 – 05/01/25

Resultado esperado: El sistema muestra total = 0 o mensaje “No hay huéspedes en las fechas seleccionadas”.

RFC3-E4 (Falla lógica) – Fechas inválidas

Datos de prueba:

- Rango: 12/10/26 – 10/10/26

Resultado esperado: El sistema rechaza la consulta y muestra error de rango inválido.

RFC4 – Consultar el ingreso total por hotel por rango de fechas

RFC4-E1 (Éxito) – Ingresos por hotel y agregado por ciudad para un año

Objetivo: Visualizar los ingresos generados por cada hotel en un año y mostrar agregados por hotel y por ciudad.

Precondiciones:

- **Ciudades:** (id_ciudad='ROM', nombre_ciudad='Roma')
- **Hoteles :** (id_hotel=683782, nombre_hotel='Palazzo Navona', id_ciudad='ROM', ...)
- **Tipo_habitaciones**
- **Reservas:**(id_reserva=920001, id_cliente=123456, id_tipo=10000, fecha_entrada='05/01/26', fecha_salida='10/01/26', precio_total=1200, estado='FINALIZADA', ...)
- Reserva cancelada :(id_reserva=920005, id_cliente=500005, id_tipo=11000, fecha_entrada='20/05/26', fecha_salida='22/05/26', precio_total=700, estado='CANCELADA', ...)

Datos de prueba:

- Año / rango: 01/01/26 – 31/12/26

Pasos:

- El administrador entra al módulo de reportes financieros.
- Selecciona el año 2026 (o el rango equivalente).
- Ejecuta el reporte “Ingreso total por hotel”.
- El sistema: Toma reservas en el rango, mapea cada reserva a su hotel (por id_tipo → Tipo_habitaciones.id_hotel), suma precio_total por hotel, agrupa además por ciudad (sumando los hoteles de la ciudad).

Resultado esperado: El sistema muestra una tabla o listado con el ingreso total por hotel y el ingreso total por ciudad.

RFC4-E2 (Éxito) – Rango parcial dentro del año

Objetivo: Validar que el reporte funciona con un rango más corto (no solo un año completo).

Datos de prueba:

- Rango: 01/01/26 – 31/03/26

Resultado esperado: Solo se suman reservas cuyo período caiga dentro del rango consultado y se recalculan agregados por hotel y ciudad para ese intervalo.

RFC4-E3 (Sin resultados) – Año/rango sin reservas

Datos de prueba:

- Rango: 01/01/25 – 31/12/25 (si no hay reservas en 2025)

Resultado esperado: Ingresos por hotel y por ciudad = 0, o mensaje “No hay ingresos en el rango seleccionado”.

RFC4-E4 (Falla lógica) – Rango de fechas inválido

Datos de prueba:

- Desde: 31/12/26
- Hasta: 01/01/26

Resultado esperado: El sistema rechaza la consulta por rango inválido.

RFC5 – Servicios y tipos de habitación más populares por sede por rango de fechas

RFC5-E1 (Éxito) – Top servicios más populares en un hotel (sede) en un rango

Objetivo: Identificar cuáles servicios son más usados en una sede (hotel) durante un rango de fechas.

Precondiciones:

- Hotel: (id_hotel=683782, nombre_hotel='Palazzo Navona', id_ciudad='ROM', ...)
- Tipos de habitación del hotel: (id_tipo=10000, id_hotel=683782, ...)
- Servicios: (id_servicio=728937, nombre='Cancelación Flexible', aplica_por='RESERVA', ...)
- Reservas (en el rango): (id_reserva=930003, id_tipo=20000, fecha_entrada='15/10/26', estado='ACTIVA', ...)
- Reserva_Servicios (selecciones): (id_reserva=930001, id_servicio=728937, cantidad=1)

Datos de prueba:

- Sede/hotel: id_hotel=683782
- Rango de fechas: 10/10/26 – 20/10/26

Pasos:

- El administrador ingresa al reporte de popularidad.
- Selecciona la sede (hotel) 683782.
- Define el rango de fechas.
- Ejecuta el reporte de “servicios más populares”.
- El sistema cuenta uso por servicio.

Resultado esperado: El sistema lista servicios ordenados de mayor a menor uso, con los datos de ejemplo, “Desayuno” debería aparecer como el más popular.

RFC5-E2 (Éxito) – Tipos de habitación más populares en un hotel (sede) en un rango

Objetivo: Identificar cuáles tipos de habitación son más reservados en una sede.

Precondiciones: Las mismas del escenario anterior (reservas con id_tipo asociados al hotel).

Datos de prueba:

- Hotel: 683782
- Rango: 10/10/26 – 20/10/26

Pasos:

- Administrador selecciona sede y rango.
- Ejecuta el reporte de “tipos de habitación más populares”.
- El sistema cuenta cuántas reservas hay por id_tipo (excluyendo canceladas si así lo definieron).

Resultado esperado: Se presenta ranking de Tipo_habitaciones por cantidad de reservas.

RFC5-E3 (Éxito) – Popularidad agregada por ciudad.

Objetivo: Ver servicios y tipos más populares sumando todos los hoteles de una ciudad (útil para estrategia global).

Precondiciones: Existe ciudad ROM con múltiples hoteles y reservas.

Datos de prueba:

- Ciudad: id_ciudad='ROM'
- Rango: 10/10/26 – 20/10/26

Pasos:

- Administrador filtra por ciudad en lugar de hotel (o selecciona “todas las sedes de la ciudad”).

Resultado esperado: ranking de servicios y tipos considerando todos los hoteles de la ciudad.

RFC5-E4 (Sin resultados) – Rango sin reservas / sin servicios seleccionados

Precondiciones:

- En el rango consultado no hay reservas, o ninguna reserva tiene servicios asociados.

Datos de prueba:

- **Hotel:** 683782
- **Rango:** 01/01/25 – 31/01/25

Resultado esperado: El sistema muestra que no hay datos para calcular popularidad (ranking vacío o mensaje informativo).

RFC5-E5 (Falla lógica) – Rango de fechas inválido

Datos de prueba:

- Desde: 20/10/26
- Hasta: 10/10/26

Resultado esperado: El sistema rechaza la consulta por rango inválido.

ENTREGA 2

1. Análisis y revisión del modelo de datos

A partir de la retroalimentación dada por el monitor en la entrega 1, llegamos a varios ajustes sobre el proyecto, específicamente en el modelo conceptual, pues en varios aspectos, los errores se corrigieron antes de entregar en el modelo relacional, pero no en el conceptual. Por esto, en esta entrega se adicionó la clase “Comodidades” en el modelo conceptual, con llave primaria el id_comodidad y con foreign key id_tipo, de tipo_habitación. En la clase Tipo_habitación, antes se tenía como llave primaria nombre_habitacion, pero al ser poco práctico para los siguientes pasos, se cambió a id_tipo, y en esta misma clase fue eliminado el atributo comodidad, pues ahora es una clase aparte. Los cambios indicados a los nombres también fueron hechos sobre el modelo relacional en el Excel. Otro cambio significativo fue en la clase Habitaciones, donde renombramos el atributo numero a numero_habitacion, y se agregó un atributo llamado “estado”, que logra identificar dentro del programa si una habitación está o no ocupada. Finalmente, la llave primaria de Habitaciones pasó de ser (numero) a (numero_habitacion, id_hotel), evitando confusión entre múltiples habitaciones de diferentes hoteles que antes podían tener la misma llave primaria.

En Excel, los cambios en las tablas normalizadas fueron resaltados con otro color, específicamente en las tablas “Habitaciones”, “Comodidades” y “Tipo_habitacion_comodidad”.

2. Implementación del esquema de la base de datos

Los archivos .sql utilizados se encuentran en el repositorio bajo el nombre “Creación hoteles sql”

3. Población de datos

Los archivos .sql utilizados se encuentran en el repositorio bajo el nombre “Población sql”

4. Diseño de sentencias SQL

En este punto, primero se desarrollaron las sentencias SQL para cada consulta, y luego se hizo una página para la visualización de cada una. Es importante tener en cuenta que, de acuerdo a las instrucciones, todas estas consultas solo pueden ser vistas y manipuladas por un usuario Administrador. Para que una persona sea considerada Administrador, debe cumplir con tener en su usuario la palabra “admin”.

Los archivos .sql están en la carpeta “Consultas sql”

RFC1- Consultar habitaciones disponibles para un intervalo de fechas en un hotel particular con ciertos criterios.

Para cumplir con esta consulta, se estableció la siguiente sentencia SQL:

```
SELECT
    TH.ID_TIPO,
    TH.NOMBRE_TIPO,
    TH.COSTO_BAJA,
    TH.COSTO_ALTA,
    TH.DIMENSIONES,
    TH.TIPO_VISTA,
    TH.CANTIDAD_CAMAS,
    COUNT(H.NUMERO_HABITACION) AS HABITACIONES_DISPONIBLES
FROM TIPO_HABITACIONES TH
JOIN HABITACIONES H
    ON TH.ID_TIPO = H.ID_TIPO AND H.ID_HOTEL = 'H007' AND H.ESTADO_HABITACION = 'ACTIVA'
WHERE TH.ID_HOTEL = 'H007'
AND NOT EXISTS (
    SELECT * FROM RESERVAS R
    WHERE R.ID_TIPO = TH.ID_TIPO AND R.ESTADO NOT IN ('CANCELADA') AND R.FECHA_ENTRADA < DATE '2026-12-01' AND R.FECHA_SALIDA > DATE '2026-11-01'
)
GROUP BY
    TH.ID_TIPO, TH.NOMBRE_TIPO, TH.COSTO_BAJA,
    TH.COSTO_ALTA, TH.DIMENSIONES, TH.TIPO_VISTA, TH.CANTIDAD_CAMAS
ORDER BY TH.NOMBRE_TIPO;
```

Aquí, se usaron tres tablas, que fueron Tipo_habitaciones, Habitaciones y Reservas. En primer lugar, se hizo un inner join de Tipo_habitaciones con Habitaciones, porque solo nos interesaban los tipos de habitación que tuvieran al menos una habitación activa asignada en el hotel de la búsqueda. Si un tipo existe en Tipo_habitaciones pero no tiene habitaciones registradas en Habitaciones, no tiene sentido mostrarlo como disponible. Luego, se usó “not exists” para excluir los tipos de habitación que tuvieran alguna reserva en el rango de fechas, para esto, se descarta una reserva si su fecha de entrada es anterior a la fecha de salida buscada y si su fecha de salida es después a la fecha de entrada buscada.

A continuación se muestra el resultado de la prueba de esta consulta desde SQL Commands.

10	FROM TIPO_HABITACIONES TH
11	JOIN HABITACIONES H
12	ON TH.ID_TIPO = H.ID_TIPO
13	AND H.ID_HOTEL = 'H007'
14	AND H.ESTADO_HABITACION = 'ACTIVA'
15	WHERE TH.ID_HOTEL = 'H007'
16	AND NOT EXISTS (
17	SELECT * FROM RESERVAS R
18	WHERE R.ID_TIPO = TH.ID_TIPO
19	AND R.ESTADO NOT IN ('CANCELADA')
20	AND R.FECHA_ENTRADA < DATE '2026-12-01'
21	AND R.FECHA_SALIDA > DATE '2026-11-01'
22	)
23	GROUP BY
24	TH.ID_TIPO, TH.NOMBRE_TIPO, TH.COSTO_BAJA,
25	TH.COSTO_ALTA, TH.COSTO_MEDIO, TH.COSTO_ALTA, TH.COSTO_MEDIO

Results	Explain	Describe	Saved SQL	History
NUMERO_HABITACION		ESTADO_HABITACION		
101		ACTIVA		
102		ACTIVA		
103		ACTIVA		
201		ACTIVA		
202		ACTIVA		
203		INACTIVA		

Y, la siguiente es un escenario probado ya desde la vista de la aplicación:

RFC1 - Habitaciones Disponibles

Consulta de habitaciones disponibles

Selecciona un hotel y el rango de fechas en que deseas hospedarte. El sistema mostrará los tipos de habitación disponibles con sus precios y características.

Hotel
Hotel Salcedo

Fecha Entrada
2026-04-27

Fecha Salida
2026-04-29

Buscar

Go Actions

Id Tipo Habitación	Tipo Habitación	Dimensiones	Tipo Vista	Cantidad Camas	Habitaciones Disponibles	Costo Temporada Alta	Costo Temporada Baja
1004	Habitacion Sencilla	25	estandar	1	2	2000	600
1003	Suite Junior	55	a monumentos	1	3	6000	850

1 - 2

Para este escenario, se buscaron las habitaciones disponibles en “Hotel Salcedo” entre el 27 y el 29 de abril de este año, y el programa mostró las dos disponibles para estas fechas. Desde el botón “actions”, el usuario puede elegir qué columnas ver.

RFC2 - Consultar reservas a través del tiempo

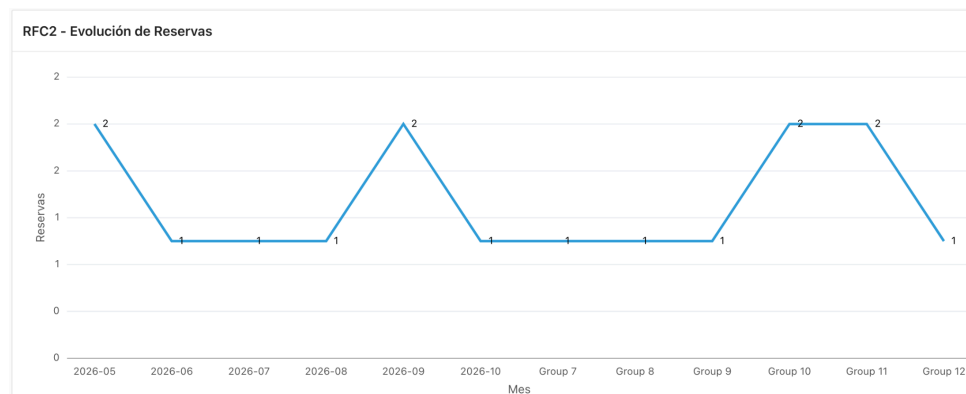
Para este requerimiento, hicimos el siguiente sql:

```
SELECT C.NOMBRE_CIUADAD, H.NOMBRE_HOTEL, TO_CHAR(R.FECHA_ENTRADA, 'YYYY-MM') AS MES, COUNT(R.ID_RESERVA) AS TOTAL_RESERVAS
FROM RESERVAS R
JOIN TIPO_HABITACIONES TH ON R.ID_TIPO = TH.ID_TIPO
JOIN HOTELES H ON TH.ID_HOTEL = H.ID_HOTEL
JOIN CIUDADES C ON H.ID_CIUADAD = C.ID_CIUADAD
WHERE R.ESTADO NOT IN ('CANCELADA')
AND R.FECHA_ENTRADA >= TO_DATE(:P3_FECHA_INICIO, 'YYYY-MM-DD')
AND R.FECHA_ENTRADA <= TO_DATE(:P3_FECHA_FIN, 'YYYY-MM-DD')
AND (:P3_ID_HOTEL IS NULL OR H.ID_HOTEL = :P3_ID_HOTEL)
AND (:P3_ID_CIUADAD IS NULL OR C.ID_CIUADAD = :P3_ID_CIUADAD)
GROUP BY C.NOMBRE_CIUADAD, H.NOMBRE_HOTEL, TO_CHAR(R.FECHA_ENTRADA, 'YYYY-MM')
ORDER BY MES, H.NOMBRE_HOTEL;
```


Los valores que comienzan por P3, son los que son parámetros en la aplicación para luego hacer la búsqueda, que reflejamos con un gráfica de línea para poder reflejar mejor los cambios en un intervalo de tiempo. Esta consulta inicia con la tabla Reservas, que tiene el atributo id_tipo que hace referencia a la tabla Tipo_habitaciones y id_hotel, que hace referencia a la tabla Hoteles. También llamamos a Ciudades, que no tiene ninguna relacion directa con las demás pero se usa para filtrar los hoteles ubicados en una ciudad específica. El primer INNER JOIN, lo usamos para asegurarnos que solo se incluyeran las reservas de habitaciones que están asociadas a un hotel válido. Luego, se usó otro INNER JOIN para que solo estuvieran las reservas de hoteles que pertenecen a una ciudad válida. Con el where, primero filtramos las reservas que no han sido canceladas, luego nos aseguramos que la fecha de entrada de la reserva debe ser mayor o igual que la fecha de inicio y menor o igual a la fecha de fin que el usuario pone. También, revisamos que el hotel de la reserva coincida con el ID del hotel buscado, y la ciudad del hotel debe coincidir con la proporcionada. Finalmente, el agrupamiento se hace con GROUP BY, específicamente a ciudad, hotel y fecha de entrada para contar las reservas en estas combinaciones.

En APEX, la gráfica, el eje x es el intervalo de tiempo, que decidimos manejar como meses, por lo que para probarla se recomienda poner fechas que tengan varios meses en medio, para poder ver la evolución real.

Por ejemplo, la siguiente es la gráfica para todos los hoteles en todas las ciudades entre abril y octubre de 2026.



RFC3: Consultar la cantidad de huéspedes por fechas y ciudad

```
SELECT C.NOMBRE_CIUDAD, H.NOMBRE_HOTEL, SUM(R.ADULTOS + R.NINOS) AS TOTAL_HUESPEDES, COUNT(R.ID_RESERVA) AS TOTAL_RESERVAS
FROM RESERVAS R
JOIN TIPO_HABITACIONES TH ON R.ID_TIPO = TH.ID_TIPO
JOIN HOTELES H ON TH.ID_HOTEL = H.ID_HOTEL
JOIN CIUDADES C ON H.ID_CIUDAD = C.ID_CIUDAD
WHERE R.ESTADO NOT IN ('CANCELADA')
AND R.FECHA_ENTRADA <= TO_DATE(:P4_FECHA_FIN, 'YYYY-MM-DD')
AND R.FECHA_SALIDA >= TO_DATE(:P4_FECHA_INICIO, 'YYYY-MM-DD')
AND C.ID_CIUDAD = :P4_ID_CIUDAD
AND (H.ID_HOTEL = :P4_ID_HOTEL)
GROUP BY C.NOMBRE_CIUDAD, H.NOMBRE_HOTEL
ORDER BY TOTAL_HUESPEDES DESC
```

Para esta consulta usamos cuatro tablas diferentes, “Reservas” que tiene la información principal de cada reserva, “Tipo_habitaciones” que permite relacionar la reserva con un hotel, “Hoteles” que da la

información del hotel y “Ciudades” que da la información de la ciudad. Usamos solo “Inner join” simples, primero, relacionando cada reserva con el tipo de habitación reservado, luego para identificar en qué hotel se hizo la reserva y finalmente para ubicar el hotel en una ciudad específica. En términos generales, en la consulta primero se suman los adultos y los niños porque una reserva puede incluir varios huéspedes. Luego se cuentan las reservas, se filtran las fechas para incluir las que se cruzan con el rango de fechas buscadas y se filtra para que se excluyan las reservas canceladas. Después, se hace un filtro opcional por hotel y se agrupa por ciudad y hotel para obtener el total de huéspedes y reservas por hotel dentro de la ciudad.

La siguiente imagen muestra un ejemplo de la cantidad de huéspedes entre abril de 2026 y abril de 2027 en Bogotá, para todos los hoteles.

Fecha Inicio

2026-04-27

Fecha Fin

2027-04-13

Id Ciudad

Bogota

Id Hotel

todos los hoteles

Buscar

Q

Go

Actions

Nombre Ciudad	Nombre Hotel	Total Huespedes	Total Reservas
Bogota	Hotel Solecitos	14	5
Bogota	Hotel Andes	10	5
Bogota	Hotel Oracle	9	4
Bogota	Hotel Salcedo	8	5

1 - 4

RFC4: Consultar el ingreso total por hotel por rango de fechas

```
SELECT
  C.NOMBRE_CIUDAD AS LABEL, SUM(R.PRECIO_TOTAL) AS VALUE
FROM RESERVAS R
JOIN TIPO_HABITACIONES TH ON R.ID_TIPO = TH.ID_TIPO
JOIN HOTELES H ON TH.ID_HOTEL = H.ID_HOTEL
JOIN CIUDADES C ON H.ID_CIUDAD = C.ID_CIUDAD
WHERE R.ESTADO NOT IN ('CANCELADA')
  AND TO_CHAR(R.FECHA_ENTRADA, 'YYYY') = :P5_ANO
GROUP BY C.NOMBRE_CIUDAD
ORDER BY SUM(R.PRECIO_TOTAL) DESC
```

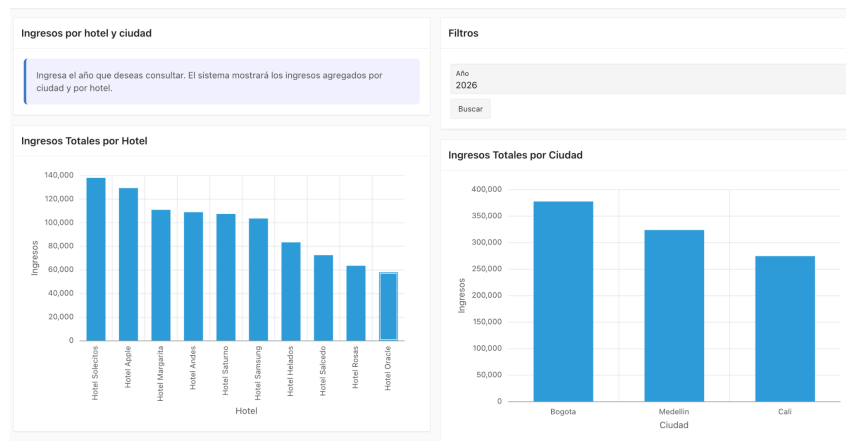
```
SELECT
  H.NOMBRE_HOTEL AS LABEL,
  SUM(R.PRECIO_TOTAL) AS VALUE
FROM RESERVAS R
JOIN TIPO_HABITACIONES TH ON R.ID_TIPO = TH.ID_TIPO
JOIN HOTELES H ON TH.ID_HOTEL = H.ID_HOTEL
JOIN CIUDADES C ON H.ID_CIUDAD = C.ID_CIUDAD
WHERE R.ESTADO NOT IN ('CANCELADA')
  AND TO_CHAR(R.FECHA_ENTRADA, 'YYYY') = :P5_ANO
GROUP BY H.NOMBRE_HOTEL
ORDER BY SUM(R.PRECIO_TOTAL) DESC
```

Para esta consulta decidimos hacer dos sentencias SQL, y dos gráficas en Apex. Las tablas usadas fueron “Reservas”, “Tipo_habitaciones”, “Hoteles”, “Ciudades”, y tres Join en cada consulta. Escogimos Inner Join para primero relacionar cada reserva con su tipo de habitación, luego identificar el hotel asociado a la reserva y ubicar el hotel dentro de una ciudad. Primero, se calculan los valores de todas las reservas para

tener el ingreso total, luego se filtra para sacar las reservas canceladas y luego se filtra por año. En las agrupaciones, para la primera consulta agrupamos todas las reservas por ciudad para ver que ciudades generan más ingresos y para la segunda, agrupamos por hotel específico, para poder ver los desempeños individuales. Ambas las ordenamos de mayor a menor ingreso.

En Apex, pusimos como valor estándar el año 2026, pues la mayoría de las reservas están centradas en este año. A continuación se pueden ver las gráficas arrojadas.

RFC4 - Ingresos por Hotel y Ciudad



RFC5: Consultar los servicios y tipos de habitación más populares por sede por rango de fechas

```
SELECT S.NOMBRE AS LABEL, SUM(RS.CANTIDAD) AS VALUE
FROM RESERVAS R
JOIN RESERVA_SERVICIOS RS ON R.ID_RESERVA = RS.ID_RESERVA
JOIN SERVICIOS S ON RS.ID_SERVICIO = S.ID_SERVICIO
JOIN TIPO_HABITACIONES TH ON R.ID_TIPO = TH.ID_TIPO
JOIN HOTELES H ON TH.ID_HOTEL = H.ID_HOTEL
WHERE R.ESTADO NOT IN ('CANCELADA')
AND H.ID_HOTEL = :P6_ID_HOTEL
AND R.FECHA_ENTRADA >= TO_DATE(:P6_FECHA_INICIO, 'YYYY-MM-DD')
AND R.FECHA_ENTRADA <= TO_DATE(:P6_FECHA_FIN, 'YYYY-MM-DD')
GROUP BY S.NOMBRE
ORDER BY SUM(RS.CANTIDAD) DESC
```

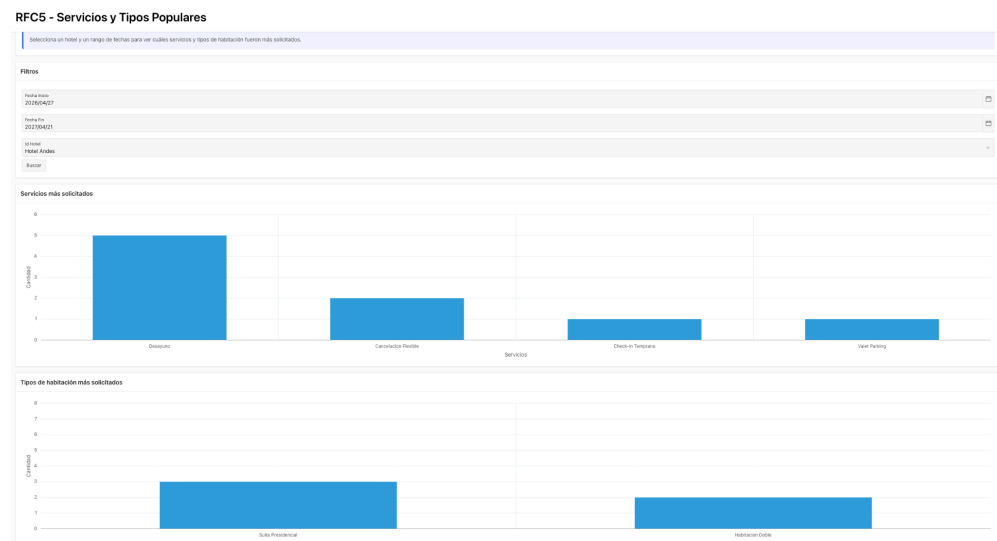
```
SELECT TH.NOMBRE_TIPO AS LABEL, COUNT(R.ID_RESERVA) AS VALUE
FROM RESERVAS R
JOIN TIPO_HABITACIONES TH ON R.ID_TIPO = TH.ID_TIPO
JOIN HOTELES H ON TH.ID_HOTEL = H.ID_HOTEL
WHERE R.ESTADO NOT IN ('CANCELADA')
AND H.ID_HOTEL = :P6_ID_HOTEL
AND R.FECHA_ENTRADA >= TO_DATE(:P6_FECHA_INICIO, 'YYYY-MM-DD')
AND R.FECHA_ENTRADA <= TO_DATE(:P6_FECHA_FIN, 'YYYY-MM-DD')
GROUP BY TH.NOMBRE_TIPO
ORDER BY COUNT(R.ID_RESERVA) DESC
```

Al igual que en la anterior consulta, en esta hicimos dos consultas. Para la primera, que tenía el fin de mostrar los servicios más utilizados, usamos las tablas “Reservas”, “Reserva_servicios”, “Servicios”, “Tipo_habitaciones” y “Hoteles”. Para la segunda, que buscaba mostrar los tipos de habitación más reservados, usamos “Reservas”, “Tipo_habitaciones” y “Hoteles”. Para consultar servicios, usamos tres Join, primero para relacionar cada reserva con los servicios consumidos, luego para identificar el nombre del servicio, también, para relacionar la reserva con el tipo de habitación y finalmente para filtrar por

sede. En la segunda consulta, solamente relacionamos la reserva con el tipo de habitación y luego filtramos por sede. En la primera consulta, se usa SUM(RS.CANTIDAD) porque un servicio puede ser usado varias veces en una misma reserva. Y en la segunda, se usa COUNT(R.ID_RESERVA) para ver cuantas veces se reservó cada tipo.

La consulta de Servicios se agrupó por tipo de servicio y el de habitación por tipo de habitación. Ambos se ordenaron de forma descendente.

En la siguiente imagen se pueden ver los servicios y las habitaciones más solicitadas para el “Hotel Andes” Entre abril de 2026 y abril de 2027.



5. Ejecución de escenarios de prueba en APEX

RF1 - Registrar Ciudad

- Interfaz con datos ingresados

DESTINA colibites_apex_admin

Registrar ciudad RF1

Registrar nueva ciudad

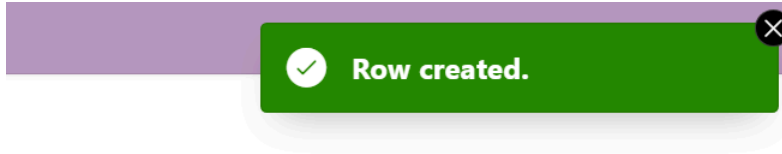
Ingresar el ID y nombre de la nueva ciudad donde existe un hotel de Darm-Alpes.

Registrar ciudad RF1

ID ciudad (ej. AMO)
PER

Nombre Ciudad
Pereira

- Respuesta del sistema



- Estado de datos en la tabla modificada.

ID_CUIDAD	NOMBRE_CUIDAD
MTR	Monteria
BMG	Bucaramanga
BOG	Bogota
MED	Medellin
CAL	Cali
PER	Pereira

6 rows returned in 0.01 seconds [Download](#)

- Descripción:
Luego de hacer la inserción de la nueva ciudad, se registra correctamente en la base de datos la nueva ciudad y su id correspondiente.

RF1 - Registrar Ciudad (Falla)

- Interfaz con datos ingresados

DESTINA

co_b165_sql_s32_admin

Registrar ciudad RF1

Registrar nueva ciudad

Ingresar el ID y nombre de la nueva ciudad donde existe un hotel de Dann-Alpes.

Registrar ciudad RF1

ID ciudad (ej. AMZ)
MTR

Nombre Ciudad
Monteria

Cancel

Create

- Respuesta del sistema

DESTINA

1 error has occurred
ORA-00001: unique constraint (CO_B165_SQL_S06.SYS_C0026986292) violated

Registrar ciudad RF1

- Estado de datos en la tabla modificada.

No se modifica.

- Descripción

Se puede notar que al tratar de ingresar una ciudad ya creada no es posible, la aplicación nos manda un mensaje de error.—

RF2 - Registrar Hotel

- Interfaz con datos ingresados

Registrar nuevo hotel

Ingresar la información del nuevo hotel de la cadena Dann-Alpes. Selecciona la ciudad donde está ubicado.

Registrar hotel - RF2

id_hotel

H0017

Nombre Hotel

Hotel el descanso

Direccion

calle 100

Telefono

3000006789

Descripcion

desc

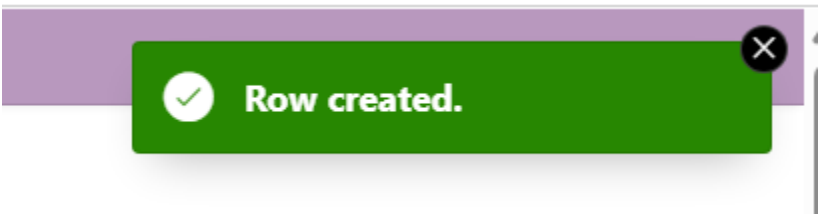
id_Ciudad

Perseira

Cancel

App 14812Page 17SessionDebugQuick EditCustomizeCreate

- Respuesta del sistema



- Estado de datos en la tabla modificada.

EDIT	ID_HOTEL	NOMBRE_HOTEL	DIRECCION	TELEFONO	DESCRIPCION	ID_Ciudad
	H016	Hotel Prueba	Calle 123	3001234567	Hotel de prueba	BOG
	H007	Hotel Salcedo	Dir 7	300000007	Desc	BOG
	H008	Hotel Margarita	Dir 8	300000008	Desc	MED
	H009	Hotel Rosas	Dir 9	300000009	Desc	CAL
	H010	Hotel Solecitos	Dir 10	300000010	Desc	BOG
	H011	Hotel Helados	Dir 11	300000011	Desc	MED
	H012	Hotel Saturno	Dir 12	300000012	Desc	CAL
	H013	Hotel Oracle	Dir 13	300000013	Desc	BOG
	H014	Hotel Apple	Dir 14	300000014	Desc	MED
	H015	Hotel Samsung	Dir 15	300000015	Desc	CAL
	H0017	Hotel el descanso	calle 100	3000006789	desc	PER

- Descripción
Se inserta correctamente el nuevo hotel asociado a la ciudad indicada

RF3 - Registrar tipo de habitación

- Interfaz con datos ingresados

Registrar Tipo Habitación - RF3

Registrar tipo de habitación

Registra un nuevo tipo de habitación para un hotel indicando sus características, costos y tipo de vista.

Registrar Tipo Habitación - RF3

id_tipo
1022

Nombre Tipo
Suite Familiar

Costo Temporada Alta
9000

Costo Temporada Baja
800

Dimensiones (m2)
90

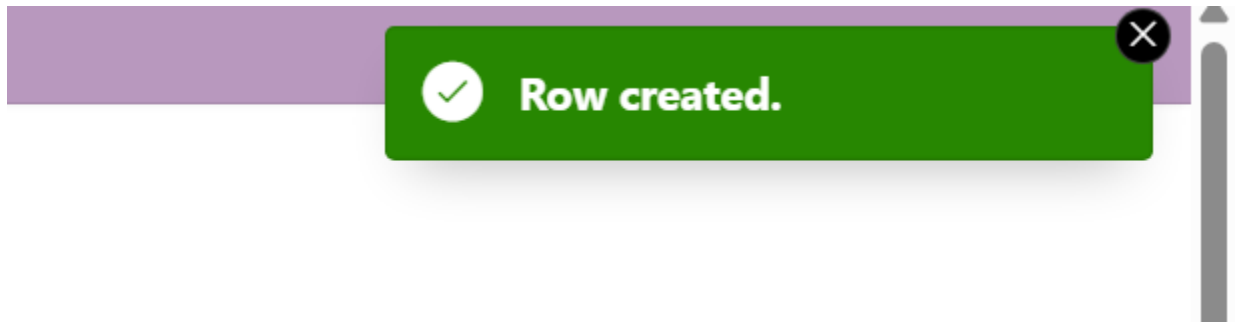
Tipo Vista
A monumentos

Cantidad Camas
4

Id Hotel
Hotel Luna

Create

- Respuesta del sistema



- Estado de datos en la tabla modificada.

EDIT	ID_TIPO	NOMBRE_TIPO	COSTO_ALTA	COSTO_BAJA	DIMENSIONES	TIPO_VISTA	CANTIDAD_CAMAS	ID_HOTEL
	1016	Habitacion Sencilla	1200	520	20	estandar	1	H013
	1017	Suite Presidencial	8200	960	88	al mar	2	H014
	1018	Habitacion Doble	3800	740	42	estandar	2	H014
	1019	Suite Junior	7000	890	65	a piscina	1	H015
	1020	Habitacion Sencilla	2200	620	28	estandar	1	H015
	1022	Suite Familiar	9000	800	90	a monumentos	4	H003
	1021	Suite Deluxe	7000	800	60	al mar	2	H001

- Descripción

Se inserta correctamente el nuevo tipo de habitación teniendo en cuenta los requerimientos de cada valor y se asocia correctamente en la base de datos con las relaciones correspondientes.

RF3 - Registrar tipo de habitación (Falla)

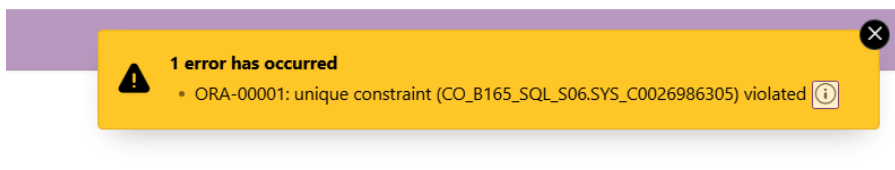
- Interfaz con datos ingresados

Registrar Tipo Habitación - RF3

id_tipo	1022
Nombre Tipo	Duplex Familiar
Costo Temporada Alta	12000
Costo Temporada Baja	800
Dimensiones (m2)	120
Tipo Vista	A monumentos
Cantidad Camas	6
Id Hotel	Hotel Margarita

Cancel Create

- Respuesta del sistema



- Estado de datos en la tabla modificada.
No se modifica
- Descripción
En este caso no es posible registrar la nueva habitación ya que en el precio supera los límites ya establecidos, entonces oracle lanza error y no permite agregar la nueva habitación a la base de datos.

RF4 - Editar Habitación

- Interfaz con datos ingresados

Editar tipo de habitación

Selecciona un tipo de habitación para editar su información. Los cambios aplican por hotel de manera independiente.

Q

Go

Actions

Id Tipo	Nombre Tipo	Costo Alta	Costo Baja	Dimensiones	Tipo Vista	Cantidad Camas	Nombre Hotel
1002	Habitacion Doble	3000	700	35	estandar	2	Hotel Andes
1021	Suite Deluxe	7000	800	60	al mar	2	Hotel Andes
1001	Suite Presidencial	8000	900	80	a piscina	2	Hotel Andes
1018	Habitacion Doble	3800	740	42	estandar	2	Hotel Apple
1017	Suite Presidencial	8200	960	88	al mar	2	Hotel Apple
1012	Habitacion Sencilla	1800	580	24	estandar	1	Hotel Helados
1011	Suite Junior	6500	870	60	al mar	1	Hotel Helados
1022	Suite Familiar	9000	800	90	a monumentos	4	Hotel Luna
1006	Habitacion Doble	4000	750	40	estandar	2	Hotel Margarita
1005	Suite Presidencial vip	8500	950	90	a piscina	2	Hotel Margarita

Registrar Tipo Habitacion - RF3

Registrar tipo de habitació

Registra un nuevo tipo de habitación para un hotel indicando sus características, costos y tipo de vista.

Registrar Tipo Habitacion - RF3

id_tipo

1018

Nombre Tipo

Habitacion Doble

Costo Temporada Alta

3800

Costo Temporada Baja

740

Dimensiones (m2)

73

Tipo Vista

Estándar

Cantidad Camas

2

Id Hotel

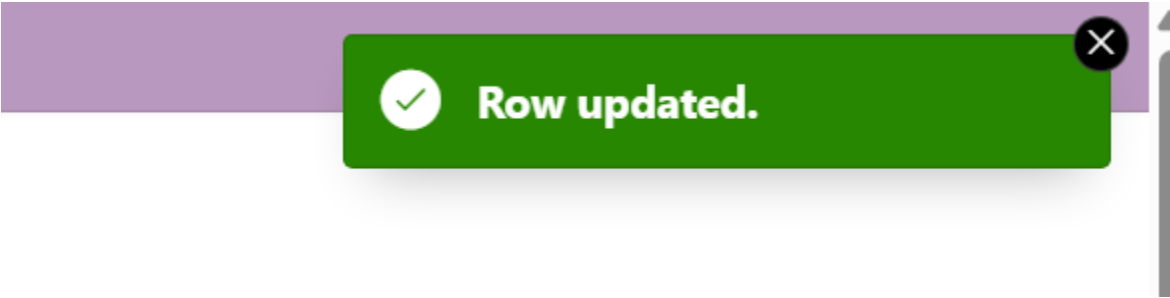
Hotel Rojas

Cancel

Delete

Apply Changes

- Respuesta del sistema



- Estado de datos en la tabla modificada.

EDIT	ID_TIPO	NOMBRE_TIPO	COSTO_ALTA	COSTO_BAJA	DIMENSIONES	TIPO_VISTA	CANTIDAD_CAMAS	ID_HOTEL
	1016	Habitacion Sencilla	1200	520	20	estandar	1	H013
	1017	Suite Presidencial	8200	960	88	al mar	2	H014
	1018	Habitacion Doble	3800	740	73	estandar	2	H006

- Descripción

Se hizo la modificación en el tipo de habitación en donde se aumentó las dimensiones y el hotel, oracle nos manda un mensaje de que se hizo la modificación correctamente y en la base de datos se ve al cambio.

RF5 - Creacion de cuenta cliente

- Interfaz con datos ingresados

Registrar Cliente RF5

Registrar nuevo cliente

Ingresar los datos del cliente para crear su cuenta en el sistema de Dann-Alpes.

DATOS DEL CLIENTE

Id Usuario

U0301

Nombre

Nombre301

Apellidos

Apellido301

Correo electrónico

correo301@mail.com

Teléfono

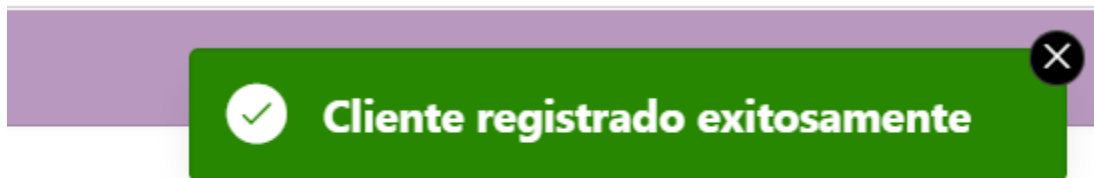
310000301

Documento de identidad

00000301

Guardar

- Respuesta del sistema



- Estado de datos en la tabla modificada.

EDIT	ID_USUARIO	NOMBRE	APELLIDOS	CORREO	TELEFONO
	U0301	Nombre301	Apellido301	correo301@mail.com	310000301
	U00011	Maria	Garcia	maria@gmail.com	3001234567

- Descripción

Al ingresar la información necesaria del nuevo cliente se registra correctamente en la base de datos.

RF6 - Gestion de cuenta Cliente.

- Interfaz con datos ingresados

En las 2 primeras imágenes se ve la modificación que se va a hacer al usuario 06, por Alexandra Gómez, el cual se hace correctamente y modifica la base de datos.

RF7 - Realizar reserva de habitación

- Interfaz con datos ingresados

Nueva Reserva

Id Reserva

R00070

Id Cliente

U0022

Hotel

Hotel Solecitos

▼

Tipo de habitacion

Suite Presidencial

▼

Fecha Entrada

4/26/2026

📅

Fecha Salida

4/29/2026

📅

Adultos

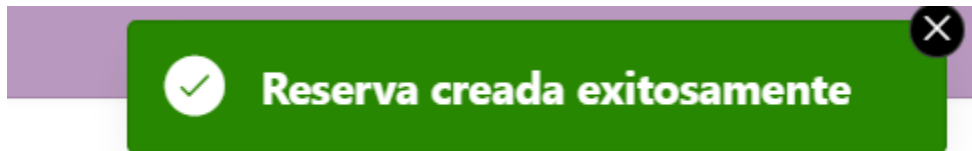
2

Niños menores de 3 años

0

Reservar

- Respuesta del sistema



- Estado de datos en la tabla modificada.

EDIT	ID_RESERVA	ID_CLIENTE	ID_TIPO	FECHA_ENTRADA	FECHA_SALIDA	ADULTOS	NINOS	PRECIO_TOTAL	ESTADO	NUMERO_NOCHES	TEMPORADA
	R00070	U0022	1009	04/26/2026	04/29/2026	2	0	2940	ACTIVA	3	BAJA
	R00021	U0001	1001	05/15/2026	05/18/2026	2	1	24000	ACTIVA	3	ALTA

- Descripción

Se genera correctamente la reserva asociada a un cliente en específico.

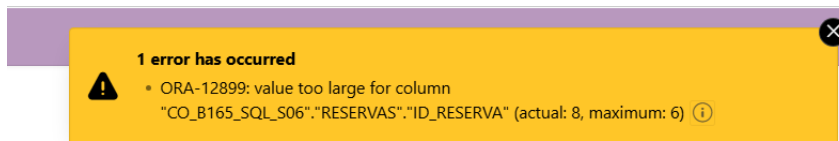
RF7 - Realizar reserva de habitación (Falla)

- Interfaz con datos ingresados

Nueva Reserva

Id Reserva	R0007000
Id Cliente	U0022
Hotel	Hotel Solecitos
Tipo de habitacion	Suite Presidencial
Fecha Entrada	4/26/2026
Fecha Salida	4/29/2026
Adultos	2
Niños menores de 3 años	0
<button>Reservar</button>	

- Respuesta del sistema



- Estado de datos en la tabla modificada.
No se modifica
- Descripción
Se puede ver que al intentar de generar una reserva con un número mucho más grande a lo establecido por la base de datos genera error y no es permitido ingresar la nueva información a la base de datos.

RF7 - Gestionar la reserva por parte del cliente

- Interfaz con datos ingresados

Gestionar Reserva - RF7B

Buscar reserva

Detalles de la reserva

Cliente

U0010

Tipo de habitación

1010

Fecha Entrada

8/12/2026

Fecha Salida

8/15/2026

Adultos

2

Niños

1

Precio Total

10500

Estado

ACTIVA

Numero Noches

3

Temporada

ALTA

Detalles de la reserva

Cliente

U0010

Tipo de habitación

1010

Fecha Entrada

7/13/2026

Fecha Salida

7/23/2026

- Respuesta del sistema



- Estado de datos en la tabla modificada.

12	R00029	U0009	1009	08/05/2026	08/10/2026	3	0	45000	ACTIVA	5	ALTA
13	R00030	U0010	1010	07/13/2026	07/23/2026	2	1	35000	ACTIVA	10	ALTA
14	R00031	U0001	1011	08/20/2026	08/24/2026	1	0	26000	ACTIVA	4	ALTA

- Descripción

En la primera imagen se ven los datos originales y a su lado por los cuales van a ser cambiados, de manera en la que se modifica la fecha de forma correcta y se actualiza la base de datos,

6. Balance del plan de pruebas

Las pruebas realizadas permitieron verificar el correcto funcionamiento de los requerimientos funcionales implementados en la aplicación desarrollada en Oracle APEX. También, se validaron operaciones de inserción, modificación y consulta, comprobando que los datos se almacenan correctamente en la base de datos en los casos exitosos. Asimismo, se realizaron pruebas de falla que permitieron evidenciar el cumplimiento de las restricciones de integridad, tales como llaves primarias, llaves foráneas y restricciones CHECK, evitando la inserción de datos inválidos. En general, el sistema respondió de manera adecuada tanto en escenarios exitosos como en escenarios de error, garantizando la consistencia y calidad de los datos.

ENTREGA 3

Actividad 1: Diseño de la base de datos

1.

a)

- Hoteles: 120
- Clientes: 800,000
- Reservas: 5,000,000
- Reseñas: 2,000,000
- Respuestas (hotel): 600,000
- Administradores: 350

Todos estos números son estimados basados en partes del enunciado. Por ejemplo, el número de reseñas crece constantemente debido a que cada reserva puede generar una opinión y no todas las reseñas reciben respuesta del hotel, por eso la entidad Respuestas es menor. Además, tuvimos en cuenta que el número de reservas históricas deben superar ampliamente el número de clientes debido a reservas repetidas (clientes que regresan).

b)

Entidades	Operación	Información Necesaria	Tipo
Hoteles	Consultar información del hotel	Datos del hotel + reputación promedio	Read
Hoteles, Reseñas	Consultar hotel con reseñas	Datos hotel + comentarios + calificaciones	Read
Reseñas, Clientes	Consultar reseñas de un hotel	Comentario + puntuación + cliente	Read
Reseñas, Respuestas	Consultar reseña con respuesta	Comentario + respuesta administrador	Read

Hoteles, Reseñas	Consultar reputación del hotel	Promedio calificaciones + cantidad reseñas	Read
Cientes	Consultar perfil cliente	Datos básicos del cliente	Read
Reseñas	Crear reseña	Calificación + comentario + fecha	Write
Respuestas	Responder reseña	Respuesta + fecha + administrador	Write
Reseñas	Moderar reseña	Estado de visibilidad/moderación	Write
Cientes	Actualizar cliente	Información del cliente	Write
Hoteles	Actualizar métricas reputación	Promedio y estadísticas	Write

c)

Entidades	Operación	Información Necesaria	Tipo	Rate
Hoteles	Consultar información del hotel	Datos hotel + reputación	Read	400/sec

Hoteles, Reseñas	Consultar hotel con reseñas	Hotel + comentarios + calificaciones	Read	250/sec
Reseñas, Clientes	Consultar reseñas de un hotel	Comentarios + cliente	Read	180/sec
Reseñas, Respuestas	Consultar reseñas con respuestas	Comentario + respuesta hotel	Read	120/sec
Hoteles, Reseñas	Consultar reputación del hotel	Promedios + estadísticas	Read	60/sec
Clientes	Consultar perfil cliente	Datos cliente	Read	20/min
Reseñas	Crear reseña	Calificación + comentario	Write	25/sec
Respuestas	Responder reseña	Respuesta administrador	Write	8/sec
Reseñas	Moderar reseña	Estado moderación	Write	3/sec
Clientes	Actualizar cliente	Datos cliente	Write	1/sec
Hoteles	Actualizar métricas reputación	Promedios y estadísticas	Write	15/sec

2.

a)

- Hoteles
- Clientes
- Reservas
- Reseñas
- Respuestas
- Administradores

b)

- Hotel — Reseña: como un hotel puede tener muchas reseñas pero una reseña solo puede tener un hotel, la relación es **1:N** (uno a muchos).
- Cliente — Reseña: un cliente puede escribir muchas reseñas pero una reseña solo puede ser de un cliente. La relación es **1:N** (uno a muchos).
- Reseña — Respuestas: Una reseña puede tener una respuesta del hotel. La cardinalidad es **1:1** y es opcional porque algunas reseñas no tienen respuestas.
- Reserva — Reseña: Una reserva puede generar una reseña. La cardinalidad es **1:1**
- Administrador — Respuestas: Un administrador puede responder muchas reseñas. La relación es entonces **1:N**

c)

Relación	Tipo	Razón principal
Hotel — Reseñas	Referenciado	Alta cardinalidad y crecimiento ilimitado
Cliente — Reseñas	Referenciado	Evitar duplicación y crecimiento
Reserva — Reseña	Embebido	Relación 1:1 y acceso conjunto
Reseña — Respuestas	Embebido	Se consultan siempre juntas

Administrador — Respuestas	Referenciado	Un administrador responde muchas reseñas
----------------------------	--------------	--

* Pusimos las guidelines de las tablas abajo tal cuál aparecen en el anexo C para no confundirnos al hacer las traducciones y que fuera más fácil comparar con el anexo.

- **Hotel - Reseña: Referenciado**

Guideline	Resultado
Query Atomicity	Sí, se consultan juntas
Go Together	Sí
Cardinality	Muy alta
Document Growth	Crecimiento ilimitado

La alta cardinalidad y crecimiento continuo hacen que embeber no sea adecuado.

- **Cliente — Reseña: Referenciado**

Guideline	Resultado
Data Duplication	Alta si se embebe
Cardinality	Alta

Individuality	La reseña puede existir separada
---------------	----------------------------------

Acá pasa lo mismo que en el anterior y por eso es referenciado

- **Reseña — Respuestas: embebido**

Guideline	Resultado
Go Together	Sí
Query Atomicity	Sí
Cardinality	Baja
Simplicity	Sí

La relación es cercana y simple y por eso es embebida

- **Reserva — Reseña: embebido**

Guideline	Resultado
Simplicity	Sí

Query Atomicity	Sí
Go Together	Sí
Cardinality	Baja

Pasa lo mismo de la anterior con la baja cardinalidad y la simplicidad y por eso es embebido.

- **Administrador — Respuestas**

Guideline	Resultado
Cardinality	Alta
Individuality	Administrador existe solo
Data Duplication	Evitar repetir datos admin

Por la alta duplicación de datos y la cardinalidad es referenciado

d)

- **Hotel - Reseña: Referenciado**

- Colección hotel

{

"_id": "HOT001",

```
"nombre": "Dann Carlton Medellín",  
"ciudad": "Medellín",  
"ratingPromedio": 4.6  
}
```

- Colección reseñas

```
{  
  "_id": "REV900",  
  "hotel_id": "HOT001",  
  "cliente_id": "CLI100",  
  "reserva_id": "RES800",  
  "calificación": 5,  
  "comentario": "Excelente servicio",  
  "fecha": "2026-05-20"  
}
```

- **Cliente - Reseñas**

- Colección Reseñas

```
{  
  "_id": "REV900",  
  "cliente_id": "CLI100",  
  "hotel_id": "HOT001",  
  "calificacion": 5,  
  "comentario": "Excelente servicio"  
}
```

- Colección Cliente

```
{  
  
  "_id": "CLI100",  
  
  "nombre": "María Pérez",  
  
  "correo": "maria@email.com"  
  
}
```

- **Reserva - Reseñas**

```
{  
  
  "_id": "RES800",  
  
  "cliente_id": "CLI100",  
  
  "hotel_id": "HOT001",  
  
  "fechaIngreso": "2026-05-10",  
  
  "reseña": {  
  
    "calificacion": 5,  
  
    "comentario": "Excelente atención",  
  
    "fecha": "2026-05-15"  
  
  }  
  
}
```

- **Reseñas - Respuestas**

```
{  
  
  "_id": "REV900",  
  
  "hotel_id": "HOT001",  
  
  "cliente_id": "CLI100",  
  
  "calificacion": 4,
```

```
"comentario": "Muy buena experiencia",  
"destacada": true,  
"respuesta": {  
  "administrador_id": "ADM10",  
  "mensaje": "Gracias por hospedarte con nosotros",  
  "fecha": "2026-05-18"  
}  
}
```

- **Administrador - Respuestas**

```
{  
  "_id": "ADM10",  
  "nombre": "Carlos Gómez",  
  "hotel_id": "HOT001"  
}  
  
{  
  "respuesta": {  
    "administrador_id": "ADM10",  
    "mensaje": "Gracias por tu comentario"  
  }  
}
```

3. Para ver el script de creación e inserción de colección dirigirse a los scripts cargados en la entrega

Actividad 2: Diseño de sentencias MongoDB

a)

Top hoteles por calificación

- Colecciones usadas: reseñas para poder sacar las calificaciones y hoteles para poder sacar los nombres de los hoteles.
- Atributos usados: en reseñas usamos hotel_id, calificación y fecha. En hoteles, usamos _id y nombre
- Explicación: para poder hacer la consulta usamos \$match para filtrar el período, \$group para calcular promedio, \$sort para ordenar, \$limit para obtener top 10 y \$lookup para traer datos del hotel.

Evolución de la reputación de un hotel en el tiempo

- Colecciones usadas: reseñas para poder sacar las calificaciones históricas.
- Atributos usados: usamos hotel_id, calificación y fecha.
- Explicación: para poder hacer la consulta usamos \$match para seleccionar hotel y año, \$group agrupando por mes, \$avg para promedio mensual y \$sort para ordenar cronológicamente.

Perfil comparativo de hoteles por ciudad

- Colecciones usadas: hoteles para filtrar ciudad y reseñas para las calificaciones y respuestas.
- Atributos usados: En hoteles usamos ciudad y nombre y en reseñas usamos hotel_id, calificación, respuestas y destacada.
- Explicación: para poder hacer la consulta usamos \$lookup para unir hoteles y reseñas, \$group para métricas, \$cond para porcentajes.

b)

Top hoteles por calificación

```
db.reseñas.aggregate([
  {
    $match: {
      fecha: { $gte: "2026-01-01", $lte: "2026-12-31" }
    }
  },
  {
    $group: {
      _id: "$hotel_id",
      calificacionPromedio: { $avg: "$calificacion" },
      totalReseñas: { $sum: 1 }
    }
  },
  {
    $sort: { calificacionPromedio: -1 }
  },
  {
    $limit: 10
  },
  {
    $lookup: {
      from: "hoteles",
      localField: "_id",
      foreignField: "_id",
      as: "hotel"
    }
  },
],
```

Resultado

```
< {
  _id: 'H010',
  calificacionPromedio: 5,
  'totalReseñas': 2,
  nombreHotel: 'Hotel Solecitos',
  ciudad: 'BOG'
}
{
  _id: 'H001',
  calificacionPromedio: 4.5,
  'totalReseñas': 2,
  nombreHotel: 'Hotel Andes',
  ciudad: 'BOG'
}
{
  _id: 'H007',
  calificacionPromedio: 4,
  'totalReseñas': 2,
  nombreHotel: 'Hotel Salcedo',
  ciudad: 'BOG'
}
{
  _id: 'H008',
  calificacionPromedio: 4,
  'totalReseñas': 1,
  nombreHotel: 'Hotel Margarita',
  ciudad: 'MED'
}
{
  _id: 'H002',
  calificacionPromedio: 3.5,
  'totalReseñas': 2,
  nombreHotel: 'Hotel Sol',
  ciudad: 'MED'
}
```

Evolución de la reputación de un hotel en el tiempo

```
> db.reservas.aggregate([{$match: { hotel_id: "HOT001"}},

  {$project: {mes: {$substr: ["$reseña.fecha", 0, 7]},
    calificacion: "$reseña.calificacion"}},

  {$group: { _id: "$mes",
    promedioMensual: {
      $avg: "$calificacion"}}},

  {$sort: {_id: 1}}])

< {
  _id: '2026-05',
  promedioMensual: 5
}
```

Perfil comparativo de hoteles por ciudad

```

> db.hoteles.aggregate([{$match: {ciudad: "Medellín" }},

{ $lookup: {from: "reseñas",
  localField: "_id",
  foreignField: "hotel_id",
  as: "reseñas"} },

{$project: {nombre: 1,
  promedioGeneral: {$avg: "$reseñas.calificacion"},
  totalReseñas: {$size: "$reseñas"},
  respuestasAdministrador: { $size:

    {$filter: {
      input: "$reseñas",
      as: "r",
      cond: {
        $ne: ["$$r.respuesta", null] }} }},

  reseñasDestacadas: { $size: {
    $filter: {input: "$reseñas",
      as: "r",
      cond: {
        $eq: ["$$r.destacada", true] }}
    }}}

  })
< {
  _id: 'HOT001',
  nombre: 'Dann Carlton Medellín',
  promedioGeneral: 4,
  'totalReseñas': 1,
  respuestasAdministrador: 1,
  'reseñasDestacadas': 1
}
{
  _id: 'HOT003',
  nombre: 'Hotel Caro',
  promedioGeneral: null,
  'totalReseñas': 0,
  respuestasAdministrador: 0,
  'reseñasDestacadas': 0
}

```

RF1:

Poniendo en el formulario de APEX:

RF1E3- Crear Reseña

RF1- Crear Reseña

H001

R00040

U0010

5

muy bueno todq

Crear Reseña

¡Reseña creada exitosamente!

Pero, si se vuelve a poner, ya no funciona:

RF1- Crear Reseña

H001

R00040

U0010

5

no me gusto

Crear Reseña

Ya existe una reseña para esta reserva

Y, en MongoDB Compass, se crea el documento de la siguiente manera:

```
_id: ObjectId('6a1511243400ee5eee711cf1')
reserva_id : "R00040"
cliente_id : "U0010"
calificacion : 5
comentario : "muy bueno todo"
hotel_id : "H001"
fecha : "2026-05-26T03:19:00.330612"
visible : true
destacada : false
votos_utilidad : 0
```

También, se puede probar con los siguientes datos, donde la reseña aún no existe pero la reserva sí:

Hotel: H002, Reserva: R00042, Cliente: U0012

Hotel: H007, Reserva: R00043, Cliente: U0013

Hotel: H008, Reserva: R00044, Cliente: U0014

Hotel: H009, Reserva: R00045, Cliente: U0015

RF2:

Para editar, por ejemplo, la anterior reseña creada, se debe insertar en el formulario los mismos datos

RF2 y RF3 Editar y Eliminar

Editar Reseña (RF2)

H001

R00040

2

emperorro

Editar Reseña

Y luego, en MongoDB, se verá el documento actualizado:

```
_id: ObjectId('6a1511243400ee5eee711cf1')
reserva_id: "R00040"
cliente_id: "U0010"
calificacion: 2
comentario: "emperoro"
hotel_id: "H001"
fecha: "2026-05-26T03:19:00.330612"
visible: true
destacada: false
votos_utilidad: 0
```

RF3:

Para eliminar alguna reseña, es suficiente con insertar los datos de ID Hotel e ID Reserva en el formulario, como se ve a continuación

Eliminar Reseña (RF3)

Eliminar Reseña

¡Reseña eliminada!

Y en Mongo ya no estará el registro que antes estaba:

```
_id: ObjectId('6a150ff83400ee5eee711cf8')
reserva_id: "R00021"
cliente_id: "U0001"
calificacion: 1
comentario: "MAL"
hotel_id: "H001"
fecha: "2026-05-26T03:09:55.458983"
visible: true
destacada: false
votos_utilidad: 0
```

```
_id: ObjectId('6a150ff83400ee5eee711cf9')
reserva_id: "R00000"
cliente_id: "CL1100"
calificacion: 3
comentario: "Medio medio"
hotel_id: "H002"
fecha: "2026-05-26T03:14:00.247080"
visible: true
destacada: false
votos_utilidad: 0
```

RF4:

Para ver reseñas, solo se necesita poner el id del hotel y el tipo de orden, y se mostrarán los resultados ordenados.

RF4 Ver Reseñas

RF4 Ver Reseñas

H010

Ordenar por Utilidad

Ver Reseñas

U0006

5/7/2026

4/5

2 útil

Muy cómodo

U0003

13/9/2026

4/5

2 útil

Excelente desayuno

U0009

14/11/2026

4/5

2 útil

Personal muy amable

RF5:

Aquí, se puede votar por una reseña como útil, para lo cual se debe poner el número de reseña y dar click en votar, por ejemplo, antes de llenar el formulario, en MongoDB se puede ver que la reserva con id R00039, del hotel H010 tiene 2 votos de utilidad, pero al marcarlo en el formulario, aumenta a 3.

1

`_id: ObjectId('6a150df59ecb1f7e0f5e90dd')`

ObjectId

2

`reserva_id: "R00039"`

String

3

`hotel_id: "H010"`

String

4

`cliente_id: "U0009"`

String

5

`calificacion: 4`

Int32

6

`comentario: "Personal muy amable"`

String

7

`visible: true`

Boolean

8

`destacada: false`

Boolean

9

`votos_utilidad: 2`

Int32

10

`fecha: "2026-11-15"`

String

CANCEL

UPDATE

Marcar Reseña como Útil (RF5)

H010

R00039

Marcar como Útil

¡Voto registrado!

```
_id: ObjectId('6a150df59ecb1f7e0f5e90dd')
reserva_id: "R00039"
hotel_id: "H010"
cliente_id: "U0009"
calificacion: 4
comentario: "Personal muy amable"
visible: true
destacada: false
votos_utilidad: 3
fecha: "2026-11-15"
```

Esto se puede probar con las siguientes reseñas activas:

Id reserva: R00024

Id hotel: H008

Id reserva: R00025

Id hotel: H009

Id reserva: R00026

Id hotel: H010

Id reserva: R00027

Id hotel: H007

Id reserva: R00029

Id hotel: H001

RF6:

En el RF6, solo poniendo el id del usuario, los demás usuarios pueden ver el historico de reseñas que ha hecho, por ejemplo, el usuario U0007 ha hecho dos:

Historial de Reseñas (RF6)

U0007

Ver Historial

Hotel: H007 | **Calificación:** 5/5 | **Estado:** Publicada | **Útil:** 8 | **Respuesta:** No
Atención impecable

Hotel: H008 | **Calificación:** 2/5 | **Estado:** Publicada | **Útil:** 0 | **Respuesta:** No
Wifi muy lento

Los siguientes clientes han hecho reservas: U0004, U0005, U0006, U0007, U0009, U0010, U0001

Escenarios de prueba

- a. *RF1 y RF2, intento de insertar a través de MongoDB Shell un documento que no cumpla con el esquema de validación*

En reseñas, según las validaciones, un campo obligatorio es la calificación. Si tratamos de poner una reseña sin este campo, saldrá error como se ve a continuación:

```
> db.resenas.insertOne({
  reserva_id: "R99999",
  hotel_id: "H001",
  cliente_id: "U0001",
  comentario: "Sin calificacion"
})
✖ ▶ MongoServerError: Document failed validation
ISIS2304A15202610 >
```

Lo mismo sucede si se pone sin comentario:

```
> db.resenas.insertOne({
  reserva_id: "R99998",
  hotel_id: "H001",
  cliente_id: "U0001",
  calificacion: 4
})
✖ ▶ MongoServerError: Document failed validation
ISIS2304A15202610 >
```

Y, para el requerimiento funcional 2, está prohibido editar una reseña para poner otra con un comentario como número, en vez de String. Esto pasa porque sigue las mismas validaciones de la colección reseña, por lo que saldrá error:

```
> db.resenas.updateOne(
  { reserva_id: "R00029" },
  { $set: { calificacion: 4, comentario: 12345 } }
)
✖ ▶ MongoServerError: Document failed validation
ISIS2304A15202610 > |
```

- b. A través APEX o de un script inserte suficientes datos para cada una de las colecciones, de tal manera que las consultas de los RF de Consulta (RFC1, RFC2 y RFC3) puedan ser probadas. En caso de haber usado un script, anéxelo a la entrega***

RFC1 – Top hoteles por calificación. Consulta los 10 hoteles con mejor calificación promedio en un período definido.

La aplicación hace el cálculo automáticamente cada vez que se necesita consultar los 10 hoteles con mejor calificación, para luego ser mostrados en la página, así como se ve en la captura.

RFC1 - Top Hoteles por Calificación

Top Hoteles por Calificación				
Posición	Hotel	Ciudad	Calificación Promedio	Total Reseñas
1	Hotel Solecitos	BOG	5.0	2
2	Hotel Salcedo	BOG	4.3	3
3	Hotel Andes	BOG	4.3	3
4	Hotel Margarita	MED	4.0	1
5	Hotel Sol	MED	3.5	2
6	Hotel Rosas	CAL	2.0	1

RFC2 – Evolución de la reputación de un hotel en el tiempo: Para un hotel seleccionado, muestra cómo ha evolucionado su calificación promedio mes a mes durante un año determinado.

Lo primero que se ve antes de hacer la consulta es insertar los datos con los que se va a hacer la búsqueda de la reputación de un hotel.

RFC2E3 - Evolución Reputación Hotel

RFC2 - Evolución Reputación Hotel

Hotel ID: Año:

Luego de hacer la búsqueda sale la calificación, las reseñas y las fechas de publicación, en caso de que no haya registros para un año en específico, sale un mensaje indicando que no existe información de acuerdo a la búsqueda.

RFC2E3 - Evolución Reputación Hotel

RFC2 - Evolución Reputación Hotel		
Hotel ID: H001	Año: 2026	Consultar
Mes	Calificación Promedio	Total Reseñas
2026-01	5.0	1
2026-03	4.0	1

RFC2 - Evolución Reputación Hotel

Hotel ID: Año:

No hay reseñas para ese hotel en ese año.

RFC3 – Perfil comparativo de hoteles por ciudad: Para una ciudad seleccionada, presenta una tabla comparativa de todos sus hoteles mostrando: calificación promedio general, total de reseñas,

porcentaje de reseñas con respuesta del administrador, y porcentaje de reseñas destacadas. Permite al administrador general identificar cuáles hoteles de una ciudad están por debajo del promedio de la ciudad.

Lo primero en salir, es la interfaz para escoger la ciudad por la que se va a comparar, en donde se ingresa el código de la ciudad, este es el resultado para BOG.

RFC3- Perfil Comparativo por Ciudad

Ciudad:

Promedio de la ciudad: 4.6

Hotel	Calificación Promedio	Total Reseñas	% con Respuesta	% Destacadas	Estado
Hotel Andes	4.3	3	33%	0%	Bajo promedio
Hotel Salcedo	4.3	3	0%	33%	Bajo promedio
Hotel Solecitos	5.0	2	0%	0%	OK

En caso de que no haya reseñas para una ciudad el resultado, será el siguiente.

RFC3- Perfil Comparativo por Ciudad

Ciudad:

No hay hoteles con reseñas en esa ciudad.

Ahora, se evidencia el script usado para poblar la base de datos, que de igual manera se encuentra en el repositorio de github.

```
db.hoteles.insertMany([
  { _id: "H001", nombre: "Hotel Andes", ciudad: "BOG", ratingPromedio: 0.0 },
  { _id: "H002", nombre: "Hotel Sol", ciudad: "MED", ratingPromedio: 0.0 },
  { _id: "H003", nombre: "Hotel Luna", ciudad: "CAL", ratingPromedio: 0.0 },
  { _id: "H007", nombre: "Hotel Salcedo", ciudad: "BOG", ratingPromedio: 0.0 },
  { _id: "H008", nombre: "Hotel Margarita", ciudad: "MED", ratingPromedio: 0.0 },
  { _id: "H009", nombre: "Hotel Rosas", ciudad: "CAL", ratingPromedio: 0.0 },
  { _id: "H010", nombre: "Hotel Solecitos", ciudad: "BOG", ratingPromedio: 0.0 }
])
```

```
db.clientes.insertMany([  
  { _id: "CLI100", nombre: "Maria Perez", correo: "maria@email.com" },  
  { _id: "CLI101", nombre: "Stefani Germanotta", correo: "stef@email.com" }  
])
```

```
db.administradores.insertMany([  
  { _id: "ADM10", nombre: "Carlos Gomez", hotel_id: "H001" },  
  { _id: "ADM11", nombre: "Juan Londoño", hotel_id: "H002" },  
  { _id: "ADM12", nombre: "Benito Martinez", hotel_id: "H007" }  
])
```

```
db.reservas.insertMany([  
  { _id: "RES800", cliente_id: "CLI100", hotel_id: "H001", fechaIngreso: "2026-05-10", estado:  
"completada" },  
  { _id: "RES801", cliente_id: "CLI101", hotel_id: "H007", fechaIngreso: "2026-05-12", estado:  
"completada" },  
  { _id: "R00028", cliente_id: "U0008", hotel_id: "H001", fechaIngreso: "2026-07-22", estado:  
"completada" },  
  { _id: "R00070", cliente_id: "U0022", hotel_id: "H002", fechaIngreso: "2026-04-26", estado:  
"completada" }  
])
```

```
db.resenas.insertMany([  
  { reserva_id: "R00024", hotel_id: "H008", cliente_id: "U0004", calificacion: 2, comentario: "Servicio  
muy lento", visible: true, destacada: false, votos_utilidad: 0, fecha: "2026-06-12" },  
  { reserva_id: "R00025", hotel_id: "H009", cliente_id: "U0005", calificacion: 5, comentario:  
"Espectacular vista", visible: true, destacada: false, votos_utilidad: 6, fecha: "2026-06-22" },  
])
```

{ reserva_id: "R00026", hotel_id: "H010", cliente_id: "U0006", calificacion: 4, comentario: "Muy cómodo", visible: true, destacada: false, votos_utilidad: 2, fecha: "2026-07-06" },

{ reserva_id: "R00027", hotel_id: "H007", cliente_id: "U0007", calificacion: 5, comentario: "Atención impecable", visible: true, destacada: true, votos_utilidad: 8, fecha: "2026-07-18" },

{ reserva_id: "R00029", hotel_id: "H001", cliente_id: "U0009", calificacion: 3, comentario: "Aceptable pero caro", visible: true, destacada: false, votos_utilidad: 1, fecha: "2026-08-10" },

{ reserva_id: "R00035", hotel_id: "H001", cliente_id: "U0005", calificacion: 5, comentario: "El mejor hotel de la ciudad", visible: true, destacada: true, votos_utilidad: 10, fecha: "2026-10-01" }

])